

A Basic Introduction to Static Program Analysis

Francesco Ranzato

University of Padova, Italy, francesco.ranzato@unipd.it

Abstract

These notes are intended to provide a (very) basic introduction to static program analysis by abstract interpretation. Some familiarity with basic order theory and denotational program semantics is required. This document is used as teaching material within the “Software Verification” course of the MSc in Computer Science, at the University of Padova, Italy.

1	Introduction to Static Program Analysis	2
1.1	What is Static Program Analysis?	2
1.2	An Introductory Example	2
1.3	Example: the Termination Problem	5
1.4	Optimizing Compilers	8
1.5	Designing Static Analyses	9
2	Introduction to Abstract Interpretation	10
2.1	Tools	10
2.2	Basic Ideas	12
2.3	Basic Definition	13
2.4	Correctness Verification	15
3	Abstract Interpretation of Arithmetic Expressions	17
3.1	Sign Analysis of Arithmetic Expressions	17
3.2	Abstract Domains	21
3.3	Galois Connections	23
3.4	Abstract Operations	25
3.5	Soundness Theorem	26
4	Abstract Interpretation of Programs	27
4.1	Collecting Denotational Semantics	27
4.2	Abstract Semantics of Expressions	29
4.3	Abstract Program Semantics	30
4.4	Sign Analysis of Programs	32
4.5	Interval Analysis of Programs	37
4.6	Widening	39
	Bibliography	42

1 Introduction to Static Program Analysis

1.1 What is Static Program Analysis?

According to Wikipedia, [Static Program Analysis](#) is the analysis of computer software performed without running any programs, as opposed to dynamic analysis, which is performed on programs during their execution. The term is usually applied to an analysis carried out by an automated tool, with human analysis called program understanding, program comprehension, or code review. In most cases the analysis is performed on some versions of the source code and in other cases by using some form of its object code.

A static analysis aims at proving with a “100% guarantee” that some programs are correct, i.e., do not have bugs. To be more precise, it makes use of some automatic techniques to deduce in the compilation phase, therefore statically, approximate information on the dynamic behaviour of the program in its execution phase.

What we want to analyze are some relevant dynamic properties of the programs, such as the values stored by variables, the properties related to program termination, or the ones related to the shape of the used memory, etc. However, there are other “easy” properties that can be statically inferred, such as the number of used variables, the number of function calls, the number of different classes that compose our program, etc. These latter type of properties can be derived through a “trivial” static analysis and some of them can also be automatically inferred by well-known automatic tools such as [Lint](#).

Practical applications of static program analysis include: bug detection, program verification, program parallelization, type inference, etc. Despite all these diverse applications, the original motivation for static analysis was not program verification or bug detection, rather to compile efficient programs, an objective also known as code optimization. Later on, static program analysis was also applied to the aforementioned application areas.

1.2 An Introductory Example

Let us consider the following factorial program written as a while loop in imperative pseudo-code:

```
Input: integer  $n$   
 $\{n = k \wedge n \geq 0\}$   
 $m := 2$   
while  $n > 1$  do  
   $m := m * n$   
   $n := n - 1$   
 $\{m = 2(k!)\}$ 
```

Observe that the accumulation variable m is initialized to 2. We expect that for the precondition $n = k \wedge n \geq 0$, if the program terminates then

the postcondition $m = 2(k!)$ holds. Informally, this means that if an input store σ satisfies the precondition, i.e. before executing the program, and the program with input σ terminates, then the postcondition holds for the final store of the program. To reason about this program, we consider its control flow graph (CFG) depicted in Figure 1, whose nodes are called program points and decorate the pseudo-code as follows:

Input: (1) n
 (2) $m := 2$
while (3) $n > 1$ **do**
 (4) $m := m * n$
 (5) $n := n - 1$
Output: (6) m

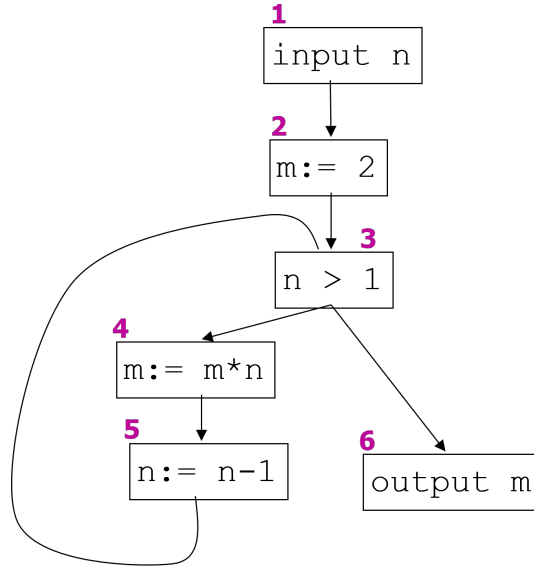


Figure 1: Control Flow Graph of the factorial program.

The nodes of the CFG contain the atomic statements of the program, namely variable assignments and Boolean guards. The crucial program point is indeed (3), namely the loop invariant program point: this is the main program property that we want to infer. Note that for the point (3) we have two incoming edges, i.e., (2) $\rightarrow$ (3) and (5) $\rightarrow$ (3).

We can infer by static analysis that at the exit point (6) the value stored in m will be even for any input value n . To conclude this, the analysis must propagate “parity information”. We therefore assign at each program variable one of these parity values:

- (i) Even: meaning that the stored value is even;
- (ii) Odd: meaning that the stored value is odd;

- (iii) DontKnow: meaning that the stored value can be either even or odd; this therefore represents lack of parity information.

Informally, we can state that we will always get an even number as output thanks to a simple inductive reasoning: at the beginning m stores 2, that, of course, is even; then, this value is updated by $m := m * n$, so that once we have an even value as base case, this even value is multiplied with any other number, so that the result stored in m will remain even. The key point in this reasoning is indeed the assignment $m := m * n$ at the program point (4). The following table describes the expected parity values for the two variables n and m at each program point:

Point	Value n	Value m	Point	Value n	Value m
(1)	DontKnow	DontKnow	(4)	DontKnow	Even
(2)	DontKnow	DontKnow	(5)	DontKnow	Even
(3)	DontKnow	Even	(6)	DontKnow	Even

Actually, for a given positive $n > 0$, this factorial program computes the double of the factorial of n , namely, $2(n!)$:

- input $n = 1 \Rightarrow$ output $m = 2$
- input $n = 2 \Rightarrow$ output $m = 4$
- input $n = 3 \Rightarrow$ output $m = 12$
- input $n = 4 \Rightarrow$ output $m = 48$
- ...

Let us consider now the “right” factorial problem with the initialization $m := 1$, that for a given positive n , computes the factorial $n!$.

Input: (1) n
 (2) $m := 1$
while (3) $n > 1$ **do**
 (4) $m := m * n$
 (5) $n := n - 1$
Output: (6) m

For this program, the previous informal analysis provides no useful information on the parity of m in (6). Here, we need to take into account the cases where n is initialized with either 0 or 1, and in these cases the final m will not be even.

Point	Value n	Value m	Point	Value n	Value m
(1)	DontKnow	DontKnow	(4)	DontKnow	DontKnow
(2)	DontKnow	DontKnow	(5)	DontKnow	DontKnow
(3)	DontKnow	DontKnow	(6)	DontKnow	DontKnow

Let us remark that this informal analysis is trivially correct, since it outputs DontKnow everywhere. Even if the input is restricted to even numbers n or to $n > 1$, the analysis cannot be precise: in (3) both n and m are DontKnow, although in (6) m will be always even.

This loss of precision is a common attribute of static analysis techniques: some property of interest actually holds but the analysis algorithm for inferring this class of properties is unable to prove it. Most interesting dynamic properties of programs are undecidable, as a consequence of Rice's Theorem, so that we cannot automatically infer them with full precision or in a reasonable time. Nevertheless, we need to ensure the correctness, also known as *soundness*, of the output of static analyses of some program property pr :

- If the program analysis of P outputs YES then the property pr surely holds for P .
- If the program analysis of P does not output YES then it may happen that the property pr does not hold for P , therefore $\neg$ YES actually means DontKnow.

1.3 Example: the Termination Problem

The [halting problem](#) in computability theory is the problem of determining, from a description of an arbitrary computer program and a given input, whether the program will finish running, or continue to run forever. Turing proved in 1936 that this problem is undecidable.

Assume that the program has as input an integer variable. As a consequence of the undecidability of the halting problem, it turns out that the set $K \triangleq \{P \in \text{Prog} \mid P(0) \text{ terminates}\}$ is not recursive.

Let us consider Figure 2, where the blue oval represents all the possible programs and the purple oval denotes the programs in K . A sound termination analysis can be done in two orthogonal ways: the set of programs which the analysis infers are terminating is determined either by the yellow area or by the green one. In the yellow case, if the analysis $\mathcal{Y}$ infers that a program P terminates on input 0 then P definitely terminates. In the green case, if the analysis $\mathcal{G}$ infers that a program P on input 0 then P may possibly terminate. Therefore, the analysis $\mathcal{Y}$ is sound for termination: $\mathcal{Y}(P) = \text{YES} \Rightarrow P$ terminates on input 0. On the other hand, the analysis $\mathcal{G}$ is sound for nontermination: $\mathcal{Y}(P) = \text{NO} \Rightarrow P$ does not terminate on input 0. In order to distinguish these

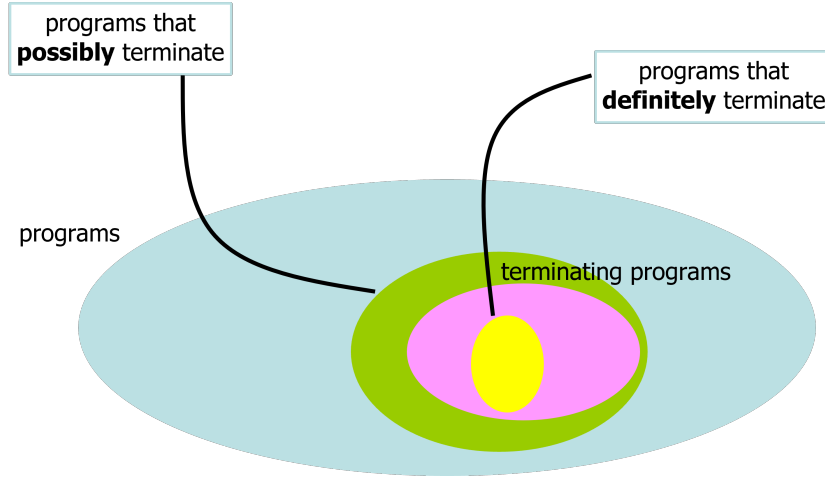


Figure 2: Sound Termination Analyses.

two orthogonal types, the first analysis is called a *definite* analysis as it provides as output programs that definitely terminate, while the second analysis is called a *possible* analysis because it computes as output programs that possibly terminate.

On the other hand, an unsound termination analysis $\mathcal{U}$ is depicted in the following Figure 3.

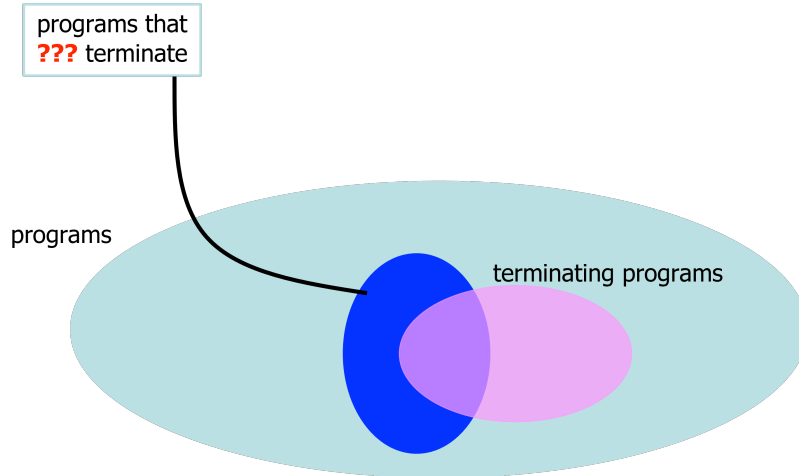


Figure 3: Unsound Termination Analysis.

This analysis $\mathcal{U}$ infers that the programs in the blue oval are terminating. However, this information is unsound: if $\mathcal{U}(P) = \text{YES}$ then P could be terminating as well as non-terminating. This means that we do not have any guarantee on the output of this analysis, and so the information inferred from $\mathcal{U}(P)$ is

useless.

To summarize, we have two types of sound termination analysis:

(A) “Definite” termination analysis:

- YES: the input program definitely terminates.
- NO: the input program could not terminate.

(B) “Possible” termination analysis:

- YES: the input program could terminate.
- NO: the input program definitely does not terminate.

This simple example shows that we need to leverage a semantics-based static analysis to provide a proof of correctness (or soundness) for the output information with respect to a ground-truth program semantics.

There exist several unsound static analysis tools such as *Coverity*. According to Wikipedia, *Coverity* is a static code analysis tool from Synopsys which enables engineers and security teams to quickly find and fix defects and security vulnerabilities in custom source code written in C, C++, Java, C#, JavaScript and more. Before its acquisition by Synopsys, Coverity was an organization founded in the Computer system Laboratory at Stanford University in Palo Alto, California. In June 2008, Coverity acquired Solidware Technologies and, in February 2014, announced an agreement to be acquired by Synopsys, an electronic design automation company, for \$350 millions. The developers of Coverity discussed their product claiming that Coverity is deliberately unsound meaning that it does not verify the absence of errors but rather tried to find as many of them as possible. However, we stress that that soundness is the indispensable property of a static analysis based on a scientific approach to static analysis and we cannot give up on it.

Example 1.3.1. Consider an hypothetical static program analysis that for detecting divisions by zero syntactically finds whether the string `/0` occurs in the program. Of course, this analysis is unsound. Let us see some instances of it.

$x = 100$ /0	$\Rightarrow$ true alarm
$\text{print}(\text{"25"} /0 9/2016)$	$\Rightarrow$ false alarm
$y = 100$ /0 2	$\Rightarrow$ false alarm
$z = 0$ $\text{print}(5/z)$	$\Rightarrow$ false negative

This analysis is unsound because false negatives may occur. □

1.4 Optimizing Compilers

Static analysis was originally conceived for designing optimizing compilers. An optimizing compiler transforms the program code to improve its (space and/or time) efficiency without affecting its input/output behaviour. Dataflow Analysis is the most used static analysis technique in optimizing compilers. Let us describe some major program transformations that might improve the code efficiency.

- (1) **Removal of common sub-expressions:** if some sub-expression is computed more than once then we can avoid re-computations. This is illustrated by the following example.

```
1 a := b * c + g;
2 d := b * c * e;
  // The code is transformed as follows
3 tmp := b * c;
4 a := tmp + g;
5 d := tmp * e;
```

- (2) **Dead-code elimination:** remove code which is unreachable. Let us see an example.

```
1 Function foo()
2   int a := 24;
3   int c;
4   c := a * 4;
5   return c;
  // Unreachable code
6   b := 24;
7   return 0;
```

- (3) **Constant folding:** if the operands of some expression E turn out to be constant then the computation of E can be done at compile-time. This is illustrated by the following example.

Example 1.4.1. With no optimization option `-O`, the gcc compiler aims at reducing the cost of compilation. When optimization flags are turned on, the compiler attempts to improve the performance and/or code size at the expense of compilation time and, possibly, threatening the chance to debug the program.

By default, the optimization level of gcc is `-O0`. Some options of optimization are as follows:

`-O1`: optimization for code size and execution time.

`-O2`: Optimize even more. gcc performs nearly all supported optimizations that do not involve a space-speed tradeoff. As compared to `-O0`, this option


```

1 int x := 14;
2 int y := 7 - x / 2;
3 return y * (28 / x + 2);
  // Propagation of x yields
4 int x := 14;
5 int y := 7 - 14 / 2;
6 return y * (28 / 14 + 2);
  // Compile-time computation yields
7 int x := 14;
8 int y := 0;
9 return 0;

```

increases both compilation time and the performance of the generated code.

-O3: Optimize yet more. **-O3** turns on all optimizations specified by **-O2** plus others.

-Os: Optimize for size. **-Os** enables all **-O2** optimizations except those that often increase code size.

-Ofast: Disregard strict standards compliance. **-Ofast** enables all **-O3** optimizations. It also enables optimizations that are not valid for all standard-compliant programs.

The manual <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> provides more details on these optimizations. □

One could ask whether *optimal* compilers exist. A *fully size-optimizing compiler* could be defined as a compiler that transforms an input program P into the program $\text{Opt}(P)$ which is “the smallest” program having the same input/output behaviour as P . Given a program Q that does not terminate on all inputs, it is simple to find $\text{Opt}(Q)$: this is the program `L1:goto L1;.` Now, assume that a fully optimizing compiler exists. We could use it to solve the halting problem: to determine if there exists an input in such that a program P terminates on input in , it is enough to check whether $\text{Opt}(P)$ coincides with `L1:goto L1;.` Since the halting problem is undecidable, it turns out that a fully optimizing compiler cannot exist.

1.5 Designing Static Analyses

To design a sound static analysis **SA** we need to:

- (1) have a reference program semantics **Sem**;
- (2) specify what and how the static analysis **SA** computes;
- (3) prove the correctness of **SA** with respect to **Sem**;

(4) efficiently implement a static analyzer for **SA**.

Static analysis techniques include:

- Abstract interpretation;
- Dataflow Analysis, aforementioned for optimizing compilers;
- Model Checking;
- Logical Deductive Systems (e.g., separation logic and SMT/SAT solvers);
- Type Systems.

In these notes, we will consider mainly abstract interpretation.

2 Introduction to Abstract Interpretation

According to [Wikipedia](#), abstract interpretation is a theory of sound approximation of the semantics of computer programs, based on monotonic functions over ordered sets, especially lattices. It can be viewed as a partial execution of a computer program which gains information about its semantics without performing all the calculations. Its main concrete application is formal static analysis. Such analyses have two main usages: (1) inside compilers to analyse programs to decide whether certain optimizations or transformations are applicable; (2) for debugging or even as certification of programs against classes of bugs.

Abstract interpretation was put forward by the French computer scientist working couple Patrick Cousot and Radhia Cousot in the late 1970s. This technique has been very influential in the fundamental and applied research on programming languages. The original paper introducing abstract interpretation has been published in the ACM Symposium on Principles of Programming Languages (POPL) conference [[Cousot and Cousot, 1977](#)], which is the top conference in programming languages, and it is the [most cited POPL paper](#) ever (POPL was established in 1973).

2.1 Tools

A lot of tools based on Abstract Interpretation have been designed, some of them are mentioned below.

[Polyspace](#), from MathWorks, is a C/C++ analyzer. A remarkable feature of this analysis tool is that is strictly based on abstract interpretation. Polyspace statically detects run-time errors, concurrency issues, security vulnerabilities, and other defects in C and C++ embedded software. Polyspace analyzes software control flow, data flow, and interprocedural behaviour.

[Astrée](#) is a static program analyzer aiming at proving the absence of runtime errors in programs written in C. It analyzes structured C programs, with complex memory usages, but without dynamic memory allocation and recursion. This encompasses many embedded programs as found in earth transportation, nuclear energy, medical instrumentation, aeronautic, and aerospace applications, in particular synchronous control/command such as electric flight control or space vessels maneuverers. Astrée is always sound: if no error is signaled, then the absence of errors has been proved.

[Interproc](#) is not a realistic analysis tool but it is good for learning abstract interpretation-based program analysis because it is available through a simple web interface. It is an interprocedural analyzer for a small imperative language with (recursive) procedure calls. It infers invariants on the numerical variables of analyzed program. It also demonstrates the features of the APRON library and the use of the Fixpoint libraries. It is implemented in OCaml. You can easily play with Interproc [online](#).

[Infer](#) is a popular analysis tool developed and maintained by Meta (former Facebook). It is a static program analyzer for Java, C, and Objective-C, written in OCaml. It is running continuously to verify selected properties of every code modification for the main Facebook apps but it may be also used for analyzing C code, and Java code that is not Android. It is open source and its source code is available on a [GitHub repository](#).

[Ikos](#) is an abstract interpretation tool designed by NASA researchers that started as a C++ library designed to facilitate the development of sound static analyzers based on abstract interpretation. Ikos also provides a C and C++ static analyzer based on LLVM and it implements scalable analyses for detecting and proving the absence of runtime errors in C and C++ programs. It is open source and the code is available on [Github](#).

[Sparta](#) is a library of software components specially designed for building high-performance static analyzers based on abstract interpretation. The purpose of SPARTA is to drastically simplify the engineering of abstract interpretation by providing a set of software components that have a simple API, are highly performant and can be easily assembled to build a production-quality static analyzer. This tool is open source and can be found on [Github](#).

In recent years, abstract interpretation has been applied to the analysis of some machine learning algorithms, such as neural networks, support vector machines, decision trees, with the goal of formally verifying their robustness properties. A major project was carried on at ETH Zurich, where some tools to be applied in *Adversarial Machine Learning* were developed. More details can be found on the [SafeAI](#) webpage and the corresponding [Github](#) repository.

One could ask why static analysis tools are not so widespread in software engineering. A possible answer is that computer engineering is the only technology field where developers are not responsible for their errors, even the trivial ones. However, we could foresee that static analysis will become more and more widespread, to the point that software manufacturers will be forced

to provide legal guarantees to the customers.

2.2 Basic Ideas

The general idea is that we are interested in providing an approximation of a concrete program property. Moreover, these approximated properties can be related through a partial order that intuitively states when a given approximated property is more precise than a second one. As an example, consider the following Figure 4 where the blue crosses denote the stores $(x, y) \in \mathbb{R}^2$ computed by a program with two floating point variables x and y . Hence, this concrete program property could be a (finite or infinite) set such as $S = \{(0, 3), (5.5, 0), (12, 7.1), \dots\}$, and this set S could not be finitely computable.

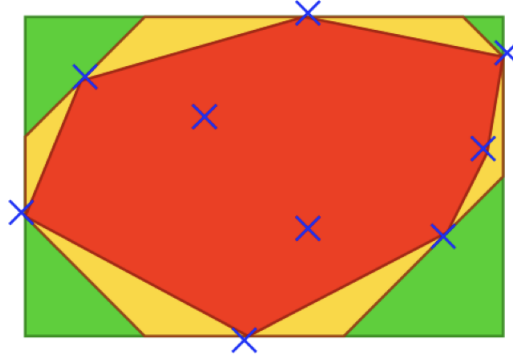


Figure 4: Approximations of a set of points in $\mathbb{R}^2$.

Several approximations of S can be taken into account, some of them are depicted in Figure 4:

Intervals: this approximation is depicted in green; for instance, an interval approximation could be $x \in [0, 12] \wedge y \in [0, 8]$. This is given by an interval $[l, u]$ for each program variable that defines its range of variation between the lower bound l and the upper bound u .

Octagons: this approximation is depicted in yellow; for instance, an octagonal approximation could be $x + y \leq 3 \wedge y \leq 0 \wedge x - y \leq -2$. This is given by a set of linear inequalities between two program variables of type $\pm x \pm y \leq k$ or by an interval of variation for a variable.

Convex Polyhedra: this approximation is depicted in red; for instance, a polyhedral approximation could be $6x - 2y \leq 3 \wedge -x - 4y \leq -1$. This is a system of linear inequalities between program variables.

It should be remarked that the reason why we may want to choose an approximation which is less precise than another one is that we need a trade-off between the cost and precision of an approximation domain. For instance, the convex polyhedra approximation is the most precise but its computation

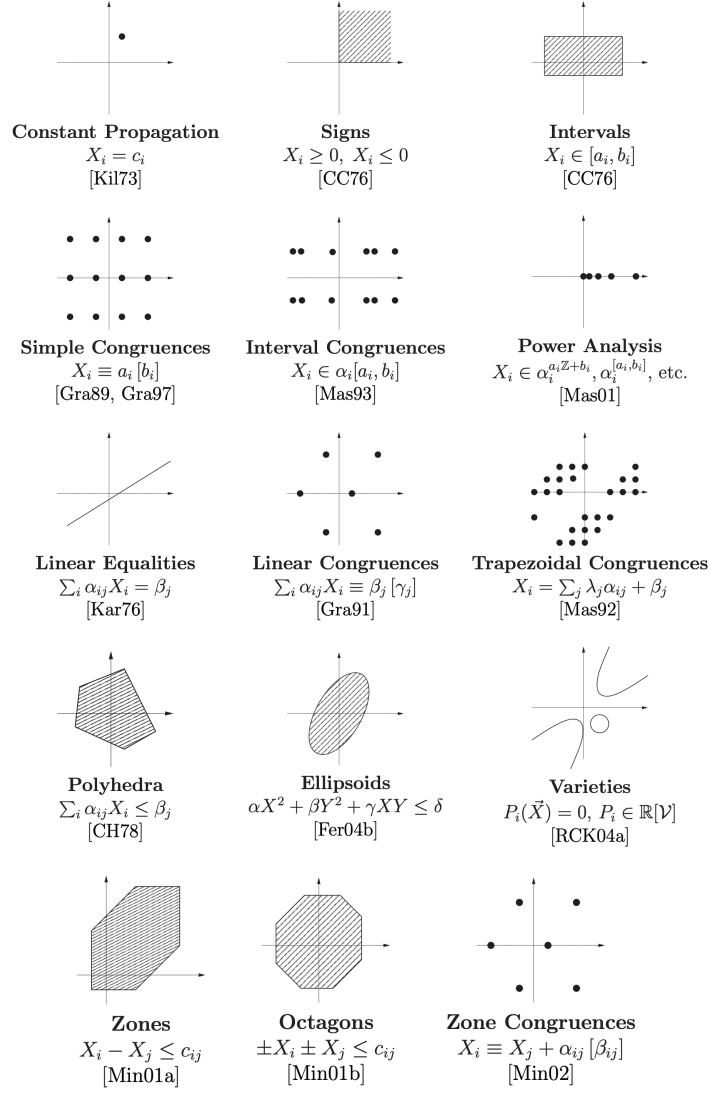


Figure 5: Numerical Abstract Domains.

needs takes exponential time w.r.t. the number of program variables, octagons are a cubic time approximation, while intervals can be computed in linear time. There are many other methods of abstracting, not only these three, as shown in Figure 5, taken from [Miné, 2004], depicts further numerical approximations used in program analysis.

2.3 Basic Definition

Abstract interpretation can be viewed as a unifying formal technique for systematically design abstractions and approximations. Moreover, in the original POPL77 paper, Cousot and Cousot [1977] stated:

“Most program analysis techniques may be understood as abstract interpretations of programs. There is a fundamental unity between all apparently unrelated program analysis techniques (flow analysis in optimizing compilers, type verification, program testing,...): a new interpretation is given to the program text which allows to built an often implicit system of equations. The problem is either to verify that a solution provided by the user is correct, or to discover or approximate such solution. The starting point is concrete semantics that provides the meaning of program commands into a given computational domain. We need also an abstract domain, which models some properties of interest of concrete computations and leaves out the remaining information. We then have an abstract semantics that allows us to ‘abstractly execute’ a program on the abstract domain in order to compute the program properties modeled by the abstract domain. The computation of the abstract semantics typically involves fixpoint computations and, when necessary, one could perform further correct approximations of the abstract semantics”.

Example 2.3.1. Let us consider the following program P :

```

1  ( $S_0$ )
   Input:  $y \in [0, 1000]$ 
2  ( $S_1$ )
3   $i := 2$ 
4  ( $S_2$ )
5  while ( $S_3$ )  $i < y$  do
6    | ( $S_4$ )
7    |  $i := i + 2$ 
8    | ( $S_5$ )
9  ( $S_6$ )

```

where $(S_i)_{i=0}^6$ denote the program points of P . Let $D \triangleq \wp(\{i, y\} \rightarrow \mathbb{Z})$. A solution $\{X_i\}_{i=0}^6 \in D^6$ of the following set of equations induced by P represents an invariant for the corresponding program points of P :

$$\begin{aligned}
X_0 &= F_0(X_0) \triangleq \{(i, y) \mid i, y \in \mathbb{Z}\} = \top \\
X_1 &= F_1(X_0) \triangleq \{(i, y) \in X_0 \mid y \in [0, 1000]\} \\
X_2 &= F_2(X_1) \triangleq \{(2, y) \mid (i, y) \in X_1\} \\
X_3 &= F_3(X_2, X_5) \triangleq X_2 \cup X_5 \\
X_4 &= F_4(X_3) \triangleq \{(i, y) \in X_3 \mid i < y\} \\
X_5 &= F_5(X_4) \triangleq \{(i + 2, y) \mid (i, y) \in X_4\} \\
X_6 &= F_6(X_3) \triangleq \{(i, y) \in X_3 \mid i \geq y\}
\end{aligned} \tag{*}$$

For any program point S_i , the most precise invariant is given by the set of states that P can compute in S_i , where, for the two integer variables of P , a state can be viewed as a pair $(i, y) \in \mathbb{Z}^2$ of integers. Hence, for point S_i , the function $F_i(X_0, \dots, X_6)$ collects all possible states in S_i . At the entry point S_0 we have no information, so that $X_0 = \mathbb{Z}^2$. For S_1 , we know that y ranges in $[0, 1000]$ and i

depends on X_0 , so that X_1 depends on X_0 . The most interesting program point is S_3 which is point of the loop invariant: X_3 is given by the union of what we get from S_2 and the contribution of each loop iteration coming from point S_5 . The above system consists of recursive equations since the information flows from each node to some other nodes. This system of equations will define our pointwise concrete collecting program semantics where each fixpoint solution provides a program invariant while the least fixpoint defines the most precise invariant.

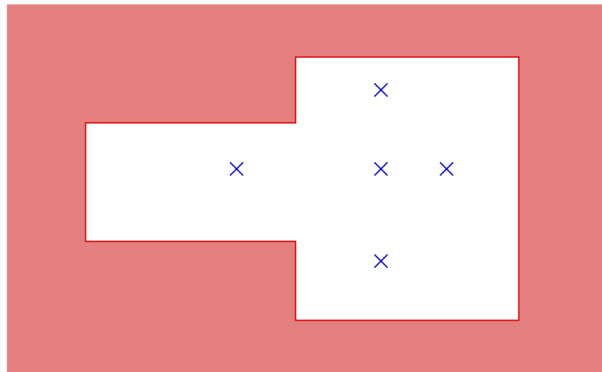
This set of concrete equations (*) is abstracted into a corresponding set of equations $X_i^\# = F_i(X_0^\#, \dots, X_6^\#)$ between abstract states $X_i^\#$ ranging in some abstract domain A where transfer functions F_i are approximated by corresponding abstract transfer functions $F_i^\#$ defined over A that simulate in a sound way the behaviour of F_i .

$$\begin{aligned} S_0^\# &= \top^\# & S_1^\# &= F_1^\#(S_0^\#) \\ S_2^\# &= F_2^\#(S_1^\#) & S_3^\# &= S_2^\# \cup^\# S_5^\# \\ S_4^\# &= F_4^\#(S_3^\#) & S_5^\# &= F_5^\#(S_4^\#) \\ S_6^\# &= F_6^\#(S_3^\#) \end{aligned}$$

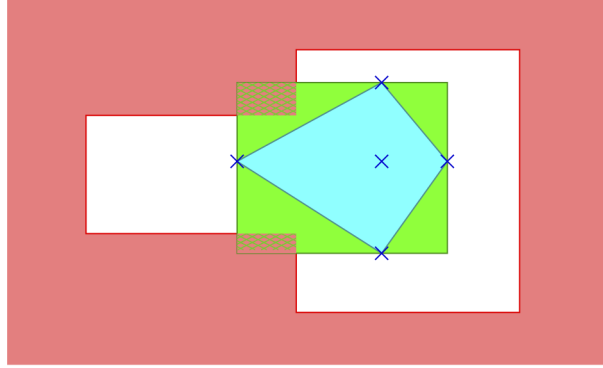
We will study how to define one such system of abstract equations and how to compute effective solutions. $\square$

2.4 Correctness Verification

Let us consider the following figure where the crosses represent the states computed by some program P , also called *reachable states*, and the red area represents the incorrect states, also called *bad states*.

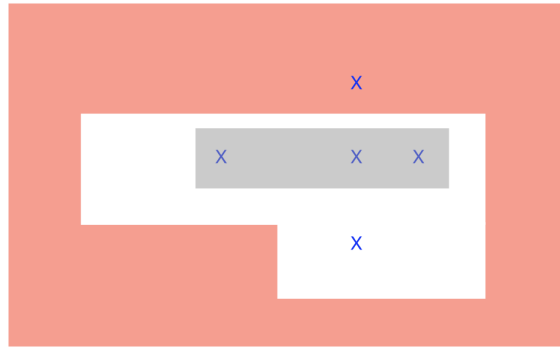


We want to verify that P is indeed correct, namely that any execution of P will never reach a bad state. This correctness verification can be performed by leveraging an over-approximation of the reachable states computed by a static analysis of P



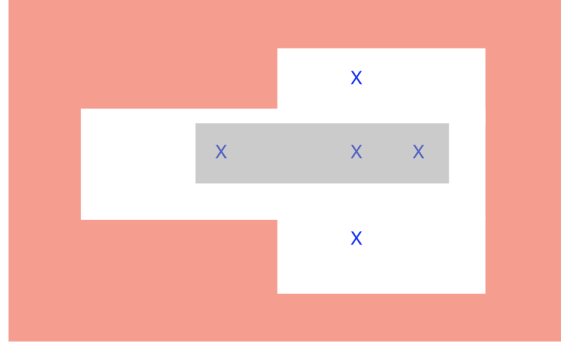
- (i) The blue area represents a (indeed the least) convex polyhedra that over-approximates the reachable states. Since the blue area does not intersect the red area, we can conclude that the program P is correct.
- (ii) The green area represents the least interval abstraction of the reachable states. In this case we cannot conclude that P is correct because the intersection of the green and red areas is not empty. Therefore, the analysis of P raises an alarm of incorrectness telling us to check the states in that nonempty intersection. However, these actually are *false alarms* because the program P is correct. This is a major issue of correctness verification based on static analyses, because if an alarm is raised then human intervention is needed to check if this is a true or false alarm.

Thus, the success or failure of verifying the correctness of P depends on the precision of the approximation of reachable states. Consider now the scenario described by the following diagram where the program P is incorrect since it can reach a bad state.



Assume that the analysis of P computes the grey area: this is an unsound analysis because the grey area is not an over-approximation of the reachable

states. A correctness verification exploiting this unsound analysis will conclude that the program P is correct, because the intersection between the grey and red areas is empty. On the other hand, if the scenario is instead given by the diagram below, then the correctness verification will infer that the program P is correct, as it is indeed the case. However, this is just a “lucky” output of this verification check. In both cases, this example shows that we cannot trust the output of an unsound static analyses.



3 Abstract Interpretation of Arithmetic Expressions

3.1 Sign Analysis of Arithmetic Expressions

Let us begin with basic integer expressions that only allow to multiply integer numbers.

$$\text{Exp} \ni e ::= n \mid e_1 * e_2$$

The concrete semantics of these expressions is given by a function $\mathcal{S} : \text{Exp} \rightarrow \mathbb{Z}$ defined as follows:

$$\begin{aligned} \mathcal{S}(n) &\triangleq n_{\mathbb{Z}} \\ \mathcal{S}(e_1 * e_2) &\triangleq \mathcal{S}(e_1) \times_{\mathbb{Z}} \mathcal{S}(e_2) \end{aligned}$$

Then, we consider an abstract semantics $\sigma : \text{Exp} \rightarrow \{-, 0, +\}$ that computes the sign of expressions.

$$\begin{aligned} \sigma(n) &\triangleq \begin{cases} - & \text{if } n_{\mathbb{Z}} < 0 \\ 0 & \text{if } n_{\mathbb{Z}} = 0 \\ + & \text{if } n_{\mathbb{Z}} > 0 \end{cases} \\ \sigma(e_1 * e_2) &\triangleq \sigma(e_1) \times^a \sigma(e_2) \end{aligned}$$

where the abstract multiplication $\times^a : \{-, 0, +\}^2 \rightarrow \{-, 0, +\}$ is defined by the following table:

$\times^a$	-	0	+
-	+	0	-
0	0	0	0
+	-	0	+

This abstract semantics σ is sound, namely, it correctly computes the sign of expressions. This proof relies on an easy structural induction on the expression e and exploits some basic properties of integer multiplications, e.g., plus times minus is minus, minus times minus is plus, etc. Thus, correctness means that for any $e \in \text{Exp}$:

$$\begin{aligned}\sigma(e) = - &\Rightarrow \mathcal{S}(e) < 0 \\ \sigma(e) = 0 &\Rightarrow \mathcal{S}(e) = 0 \\ \sigma(e) = + &\Rightarrow \mathcal{S}(e) > 0\end{aligned}$$

Actually, in this case, it turns out that an equivalence holds, so that $\Leftrightarrow$ can replace $\Rightarrow$.

Let us define a function $\gamma : \{-, 0, +\} \rightarrow \wp(\mathbb{Z})$ mapping each abstract symbol in $\{-, 0, +\}$ to the set of concrete values that it represents:

$$\begin{aligned}\gamma(-) &\triangleq \{x \in \mathbb{Z} \mid x < 0\} \\ \gamma(0) &\triangleq \{0\} \\ \gamma(+) &\triangleq \{x \in \mathbb{Z} \mid x > 0\}\end{aligned}$$

γ is called *concretization function* and each abstract symbol $a \in \{-, 0, +\}$ represents a concrete property of integers as given by $\gamma(a) \subseteq \mathbb{Z}$. The scenario can be summarized by the following diagram where $\{\cdot\} : \mathbb{Z} \rightarrow \wp(\mathbb{Z})$ denotes the singleton function mapping each $z \in \mathbb{Z}$ to $\{z\}$:

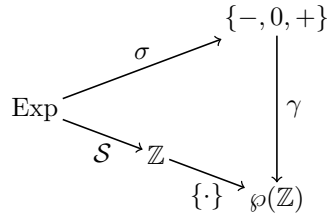


Figure 6: Concretization function.

With reference to the diagram in Figure 6, the abstract semantics σ turns out to be correct, namely σ provides a sound over-approximation of the concrete semantics $\mathcal{S}$. This means that the following inclusion holds:

$$\{\mathcal{S}(e)\} \subseteq \gamma(\sigma(e))$$

Thus, for any expression e , the concrete meaning of the abstract semantics $\sigma(e)$ is a set of integers that must include the concrete semantics $\mathcal{S}(e)$.

3.1.1 Adding Opposites

Let us add the unary operator that changes the sign of an expression to its opposite:

$$\text{Exp} \ni e ::= n \mid e_1 * e_2 \mid -e$$

so that concrete and abstract semantics are extended as follows:

$$\begin{aligned}\mathcal{S}(-e) &\triangleq -\mathcal{S}(e) \\ \sigma(-e) &\triangleq -^a \sigma(e)\end{aligned}$$

where $-^a : \{-, 0, +\} \rightarrow \{-, 0, +\}$ is defined by:

$$-^a(-) \triangleq +, \quad -^a(0) \triangleq 0, \quad -^a(+) = -.$$

It turns out that this extension preserved the soundness of the abstract semantics σ .

3.1.2 Adding Additions

The addition operation

$$\text{Exp} \ni e ::= n \mid e_1 * e_2 \mid -e \mid e_1 + e_2$$

needs some care because the abstract domain is not closed with respect to this operation. We define

$$\begin{aligned}\mathcal{S}(e_1 + e_2) &\triangleq \mathcal{S}(e_1) + \mathcal{S}(e_2) \\ \sigma(e_1 + e_2) &\triangleq \sigma(e_1) +^a \sigma(e_2)\end{aligned}$$

and then it is not clear how to fill the question marks ? in this table that sketches a definition of the abstract addition:

$+^a$	$-$	0	$+$
$-$	$-$	$-$	?
0	$-$	0	$+$
$+$	?	$+$	$+$

The issue here is which abstract value in $\{-, 0, +\}$ should be used for the sum of two integer numbers having opposite sign. Of course, the sum of two integers can be positive (e.g., $-2 + 4$), negative (e.g., $-4 + 2$), or zero (e.g., $-2 + 2$). Thus, in order to define a sound over-approximating abstract addition we need a novel abstract value $\top$ that represents any integer number, that is, such that $\gamma(\top) \triangleq \mathbb{Z}$. Hence, $\top$ means that we have no sign information. The table defining $+^a$ can be therefore extended as follows:

$+^a$	$-$	0	$+$	$\top$
$-$	$-$	$-$	$\top$	$\top$
0	$-$	0	$+$	$\top$
$+$	$\top$	$+$	$+$	$\top$
$\top$	$\top$	$\top$	$\top$	$\top$

Since the abstract domain includes a new element $\top$, we need to extend all the abstract operations to this new element as follows:

$\times^a$	$-$	0	$+$	$\top$
$-$	$+$	0	$-$	$\top$
0	0	0	0	0
$+$	$-$	0	$+$	$\top$
$\top$	$\top$	0	$\top$	$\top$

$$-^a(-) \triangleq +, \quad -^a(0) \triangleq 0, \quad -^a(+) = -, \quad -^a(\top) = \top.$$

It is worth remarking that $0 \times^a \top = 0 = \top \times^a 0$: this definition is correct and more precise than defining $0 \times^a \top \triangleq \top$.

Example 3.1.1. A loss of information in the abstract semantics may happen, as in the following case:

$$\begin{aligned} \mathcal{S}((1 + 2) - 3) &= 0 \\ \sigma((1 + 2) + (-3)) &= (+ +^a +) +^a - = + +^a - = \top. \end{aligned}$$

On the other hand, the abstract computation is as precise as possible, i.e., the abstract semantics σ turns out to be *complete*:

$$\begin{aligned} \mathcal{S}((5 * 4) + 6) &= 26 \\ \sigma((5 * 4) + 6) &= (+ \times^a +) +^a + = + +^a + = +. \end{aligned} \quad \square$$

3.1.3 Adding Divisions

Integer division does not raise issues except for the case of division by zero. As no division by zero yields a number, the result of dividing a set of integer numbers by zero is the empty set, which therefore represents a run-time error. Thus, we need to add to our abstract domain a further new element $\perp$ which represents the empty set, i.e., $\gamma(\perp) = \emptyset$. Hence, $\perp$ means termination with an error (and, later, will also represent nontermination). The abstract operations are therefore defined and extended as follows.

$/^a$	$-$	0	$+$	$\top$	$\perp$
$-$	$\top$	0	$\top$	$\top$	$\perp$
0	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$+$	$\top$	0	$\top$	$\top$	$\perp$
$\top$	$\top$	0	$\top$	$\top$	$\perp$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

$$\begin{aligned}
\perp +^a a &\triangleq \perp & a +^a \perp &\triangleq \perp \\
\perp \times^a a &\triangleq \perp & a \times^a \perp &\triangleq \perp \\
-^a(\perp) &\triangleq \perp
\end{aligned}$$

Example 3.1.2. To take into account errors generated by division by zero, we consider the concrete semantics $\mathcal{S}^c : \text{Exp} \rightarrow \{\{z\} \mid z \in \mathbb{Z}\} \cup \{\emptyset\}$ defined as follows:

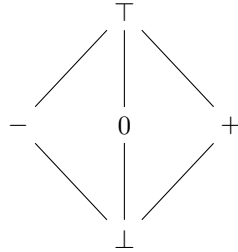
$$\mathcal{S}^c(e) \triangleq \begin{cases} \{\mathcal{S}(e)\} & \text{if } \mathcal{S}(e) \in \mathbb{Z} \\ \emptyset & \text{otherwise} \end{cases}$$

Hence, we have the following examples:

$$\begin{aligned}
\mathcal{S}^c(2/(3-1)) &= \{1\} \subseteq \gamma(\sigma(2/(3-1))) = \gamma(+/^a \top), \\
\mathcal{S}^c(2/(3-3)) &= \emptyset \subseteq \gamma(\sigma(2/(3-3))) = \gamma(+/^a \top), \\
\mathcal{S}^c(2/(3-5)) &= \{-1\} \subseteq \gamma(\sigma(2/(3-5))) = \gamma(+/^a \top). \quad \square
\end{aligned}$$

3.2 Abstract Domains

The set of symbolic values $\text{Sign} \triangleq \{\perp, -, 0, +, \top\}$ that we incrementally defined in Section 3.1 is called *abstract domain*. An abstract domain is endowed with a partial order relation $\leq$ modelling the notion of relative precision between its elements, called abstract values, where $a \leq a'$ intuitively means that a is more precise than a' , that is, the concrete property represented by a is stronger than that represented by a' . The abstract values $-, 0, +$ are not comparable with each other; $\perp$ represents the empty set, i.e. the strongest integer property, and therefore is the bottom of the abstract domain; $\top$ represents the weakest integer property, i.e. the whole set $\mathbb{Z}$, so that it is the top of the abstract domain. The Hasse diagram of $\langle \text{Sign}, \leq \rangle$ is therefore as follows:



The partial order must be coherent with the concretization map, namely the following equivalence must hold:

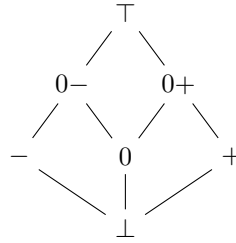
$$a \leq a' \iff \gamma(a) \subseteq \gamma(a').$$

Let us observe that Sign is a (finite) lattice since all lub's and glb's exist. We define a function $\alpha : \wp(\mathbb{Z}) \rightarrow \text{Sign}$, called *abstraction map*, which is the counterpart of γ and, intuitively, maps a set S of integers into the most precise abstract value in Sign , w.r.t. the partial order $\leq$, that represents S . For all $S \in \wp(\mathbb{Z})$,

$$\alpha(S) \triangleq \begin{cases} \perp & \text{if } S = \emptyset \\ - & \text{if } S \neq \emptyset, S \subseteq \{z \in \mathbb{Z} \mid z < 0\} \\ 0 & \text{if } S = \{0\} \\ + & \text{if } S \neq \emptyset, S \subseteq \{z \in \mathbb{Z} \mid z > 0\} \\ \top & \text{otherwise} \end{cases}$$

The relationship between the concretization map γ and the abstraction map α will be formalized as a **Galois Connection**, that will guarantee that the most precise abstract value representing a given concrete property always exists.

Example 3.2.1. Let us the following Hasse diagram defining an abstract domain denoted by Sign_+ :



Notice that $\langle \text{Sign}_+, \leq \rangle$ is a lattice. The concretization map $\gamma : \text{Sign}_+ \rightarrow \wp(\mathbb{Z})$ is defined as follows:

$$\begin{aligned} \gamma(\perp) &\triangleq \emptyset, & \gamma(+) &\triangleq \{y \in \mathbb{Z} \mid y > 0\}, \\ \gamma(0+) &\triangleq \{y \in \mathbb{Z} \mid y \geq 0\}, & \gamma(0) &\triangleq \{0\} \\ \gamma(-) &\triangleq \{y \in \mathbb{Z} \mid y < 0\}, & \gamma(0-) &\triangleq \{y \in \mathbb{Z} \mid y \leq 0\}, & \gamma(\top) &\triangleq \mathbb{Z}. \end{aligned}$$

On the other hand, the abstraction map $\alpha : \wp(\mathbb{Z}) \rightarrow \text{Sign}_+$ is defined by leveraging γ as follows:

$$\alpha(S) \triangleq \wedge \{a \in \text{Sign}_+ \mid S \subseteq \gamma(a)\}$$

For instance, α and γ act as follows:

$$\begin{aligned}
\alpha(\{0, 1, 3\}) &= 0+ \\
\gamma(0+) &\supseteq \{0, 1, 3\}, \{2, 3, 5\}, \{z \in \mathbb{Z} \mid z > 2\} \\
\gamma(\alpha(\{0, 1, 3\})) &\supseteq \{0, 1, 3\} \\
\alpha(\gamma(0+)) &= 0+
\end{aligned}
\quad \square$$

3.3 Galois Connections

A *Galois connection* (GC) is a correspondence between two posets finding several applications both in mathematics and computer science. GCs generalize the fundamental theorem of Galois theory about the correspondence between subgroups and subfields, discovered by Évariste Galois.

Definition 3.3.1. A tuple (α, C, A, γ) is a Galois connection when:

- (i) $\langle C, \leq_C \rangle$ and $\langle A, \leq_A \rangle$ are posets;
- (ii) $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ are monotone functions;
- (iii) $\forall c \in C. c \leq_C \gamma(\alpha(c))$;
- (iv) $\forall a \in A. \alpha(\gamma(a)) \leq_A a$. $\square$

In abstract interpretation terminology, C is the concrete domain, A is the abstract domain, α is the abstraction map, γ is the concretization map. The intuition from an abstract interpretation perspective is as follows: Point (ii) means that α and γ preserve their precision ordering relations, respectively, $\leq_C$ and $\leq_A$; Point (iii) means that $\alpha(c)$ is a correct abstract approximation in A for the concrete value c ; Point (iv) means that $\gamma(a)$ is correctly approximated in A by a .

3.3.1 Adjunctions

It turns out that Galois connections are equivalent to *adjunctions*, which are defined as follows.

Definition 3.3.2. A tuple (α, C, A, γ) is an adjunction when:

- (a) $\langle C, \leq_C \rangle$ and $\langle A, \leq_A \rangle$ are posets;
- (b) $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$;
- (c) $\forall c \in C. \forall a \in A. \alpha(c) \leq_A a \iff c \leq_C \gamma(a)$. $\square$

It is worth remarking that this definition does not require that α and γ are monotonic functions. The key condition here is (c) stating that the two relations $\alpha(c) \leq_A a$ and $c \leq_C \gamma(a)$ are equivalent. The intuition is that if we want to compare $c \in C$ with some $a \in A$ for inferring whether a is an approximation in A of c , then it is equivalent comparing them either in the

concrete domain C through $c \leq_C \gamma(a)$ or in the abstract domain A through $\alpha(c) \leq_A a$.

Theorem 3.3.3. (α, C, A, γ) is a GC if and only if (α, C, A, γ) is an adjunction.

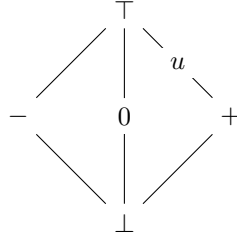
3.3.2 Galois Insertions

Definition 3.3.4. A Galois connection (α, C, A, γ) is called *Galois insertion* (GI) if one of the following equivalent conditions holds:

- (1) α is surjective.
- (2) γ is injective.
- (3) $\forall a \in A. \alpha(\gamma(a)) = a$. □

From the abstract interpretation viewpoint, a GC is indeed a GI when the abstract domain contains no “useless” abstract value. In fact, if α is not surjective then there is some abstract element $u \in A$ which is not reachable by α , so that u is useless. Equivalently, if γ is not injective then we have at least two different abstract values that represent the same concrete value, so that one of these abstract values would be enough.

Example 3.3.5. Consider the following abstract domain Sign_u for sign analysis:



whose concretization map $\gamma : \text{Sign}_u \rightarrow \wp(\mathbb{Z})$ is as follows:

$$\begin{aligned}
 \gamma(\perp) &\triangleq \emptyset & \gamma(u) &\triangleq \{y \in \mathbb{Z} \mid y > 0\} \\
 \gamma(0) &\triangleq \{0\} & \gamma(+) &\triangleq \{y \in \mathbb{Z} \mid y > 0\} \\
 \gamma(-) &\triangleq \{y \in \mathbb{Z} \mid y < 0\} & \gamma(\top) &\triangleq \mathbb{Z}
 \end{aligned}$$

It turns out that Sign_u is defined by a GC which is not a GI. The abstract values u and $+$ represent the same concrete value, so one of them is useless and could be safely removed. □

We can transform any GC (α, C, A, γ) into a GI by removing its “useless” abstract values. More precisely, this is achieved by considering a “reduced”

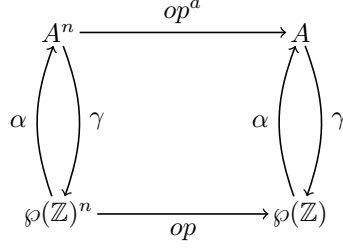


Figure 7: Concrete and abstract operations.

abstract domain $A^{\text{red}} \triangleq \alpha(C)$ and the corresponding restrictions on A^{red} of the abstraction and concretization maps:

$$\begin{aligned} \alpha^{\text{red}} : C &\rightarrow A^{\text{red}} \text{ defined by } \alpha^{\text{red}}(c) \triangleq \alpha(c) \\ \gamma^{\text{red}} : A^{\text{red}} &\rightarrow C \text{ defined by } \gamma^{\text{red}}(a) \triangleq \gamma(a) \end{aligned}$$

It turns out that $(\alpha^{\text{red}}, C, A^{\text{red}}, \gamma^{\text{red}})$ is a GI which has the same “expressive power” of the original GC, namely, $\alpha^{\text{red}}(c) = \alpha(c)$ holds.

Let us highlight that if the abstract domain is defined by a Galois connection, then we have two equivalent ways of stating that the abstract semantics is sound because

$$\text{for all } e \in \text{Exp}, \quad \{\mathcal{S}(e)\} \subseteq \gamma(\sigma(e)) \iff \alpha(\{\mathcal{S}(e)\}) \leq \sigma(e)$$

holds.

3.4 Abstract Operations

The diagram in Figure 7 considers a concrete n -ary operation op that is considered as a function on sets of integers rather than integers. This is commonly defined by the so-called *collecting* lifting of a given operation $o : \mathbb{Z}^n \rightarrow \mathbb{Z}$ which is defined as follows:

$$op(X_1, \dots, X_n) \triangleq \{o(z_1, \dots, z_n) \in \mathbb{Z} \mid \forall i \in [1, n]. z_i \in X_i\}.$$

Hence, the collective lifting of an integer operation o allows us to view that operation as a transformer of concrete integer properties. Then, we consider a domain A that abstracts $\wp(\mathbb{Z})$ through a GC, so that the Cartesian product A^n provides a pointwise abstraction of $\wp(\mathbb{Z})^n$ through an abstraction α and a concretization γ . Finally, we consider an abstract operation op^a that should soundly approximate on A the concrete behaviour of op .

More in general, we consider a concrete n -ary operation $op : C^n \rightarrow C$, which is assumed to be monotonic, and a corresponding abstract n -ary operation $op^a : A^n \rightarrow A$ which is also required to be monotonic.

Definition 3.4.1. The abstract operation op^a is a *correct approximation* of op when for all $\langle a_1, \dots, a_n \rangle \in A^n$,

$$op(\gamma(a_1), \dots, \gamma(a_n)) \leq_C \gamma(op^a(a_1, \dots, a_n)). \quad \square$$

Lemma 3.4.2. op^a is a correct approximation of op if and only if one of the following equivalent conditions holds:

- (1) For all $\langle c_1, \dots, c_n \rangle \in C^n$, $\alpha(op(c_1, \dots, c_n)) \leq_A op^a(\alpha(c_1), \dots, \alpha(c_n))$;
- (2) For all $\langle c_1, \dots, c_n \rangle \in C^n$, $op(c_1, \dots, c_n) \leq_C \gamma(op^a(\alpha(c_1), \dots, \alpha(c_n)))$;
- (3) For all $\langle a_1, \dots, a_n \rangle \in A^n$, $\alpha(op(\gamma(a_1), \dots, \gamma(a_n))) \leq_A op^a(a_1, \dots, a_n)$.

Correctness guarantees that the output of applying an abstract operation is a sound approximation of the result of applying the corresponding concrete operation. For any concrete operation op , one can always define the so-called *best correct approximation* op^A of op on the abstract domain A as follows: for all $a_i \in A$,

$$op^A(a_1, \dots, a_n) \triangleq \alpha(op(\gamma(a_1), \dots, \gamma(a_n))).$$

Definition 3.4.3. The abstract operation op^a is a *complete approximation* of op when for all $\langle c_1, \dots, c_n \rangle \in C^n$,

$$\alpha(op(c_1, \dots, c_n)) = op^a(\alpha(c_1), \dots, \alpha(c_n)). \quad \square$$

Thus, by Lemma 3.4.2, completeness is a stronger requirement than correctness for abstract operations. The intuition is that if op^a turns out to be complete then op^a is able to mimic on A the behaviour of the concrete operation op with no loss of precision. For example, the abstract multiplication $\times^a$ on Sign , as defined in Section 3.1.2, turns out to be a complete abstract operation, while the abstract addition is, of course, correct but incomplete, as shown by the following counterexample:

$$\begin{aligned} \alpha(\{2\}) +^a \alpha(\{-1\}) &= + +^a - = \top \quad \text{whereas} \\ \alpha(\{2\} +^c \{-1\}) &= \alpha(\{1\}) = +. \end{aligned}$$

3.5 Soundness Theorem

Let us prove that the concrete semantics of arithmetic expressions is correctly approximated by the above defined abstract semantics.

Theorem 3.5.1 (Soundness). *For all $e \in \text{Exp}$,*

$$\{\mathcal{S}(e)\} \subseteq \gamma(\sigma(e))$$

or, equivalently,

$$\alpha(\{\mathcal{S}(e)\}) \leq \sigma(e).$$

Proof. Let op^s be a binary (the proof for a generic n -ary operation is analogous) syntactic operation, that is, using infix notation, $e_1 op^s e_2 \in \text{Exp}$. Let $op : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ be the corresponding binary integer operation. Let $op^c : \wp(\mathbb{Z}) \times \wp(\mathbb{Z}) \rightarrow \wp(\mathbb{Z})$ be the collecting lifting of op , that is,

$$op^c(S_1, S_2) \triangleq \{op(z_1, z_2) \in \mathbb{Z} \mid z_1 \in S_1, z_2 \in S_2\}$$

Observe that op^c is monotonic. Let $op^a : A \times A \rightarrow A$ be a correct approximation of op^c on the abstract domain A . We proceed by structural induction on $e \in \text{Exp}$.

Basis. We have that $\{\mathcal{S}(n)\} = \{n_{\mathbb{Z}}\} \subseteq \gamma(\alpha(\{n_{\mathbb{Z}}\})) = \gamma(\sigma(n))$, by Galois connection.

Inductive step. We have that

$$\{\mathcal{S}(e_1 \text{ op}^s e_2)\} = \{\mathcal{S}(e_1) \text{ op } \mathcal{S}(e_2)\} = \{\mathcal{S}(e_1)\} \text{ op}^c \{\mathcal{S}(e_2)\}.$$

By inductive hypothesis on e_1 and e_2 , we have that

$$\{\mathcal{S}(e_i)\} \subseteq \gamma(\sigma(e_i)).$$

By monotonicity of op^c , we have that

$$\{\mathcal{S}(e_1)\} \text{ op}^c \{\mathcal{S}(e_2)\} \subseteq \gamma(\sigma(e_1)) \text{ op}^c \gamma(\sigma(e_2)).$$

By correctness of op^a ,

$$\gamma(\sigma(e_1)) \text{ op}^c \gamma(\sigma(e_2)) \subseteq \gamma(\sigma(e_1) \text{ op}^a \sigma(e_2)).$$

Therefore,

$$\{\mathcal{S}(e_1)\} \text{ op}^c \{\mathcal{S}(e_2)\} \subseteq \gamma(\sigma(e_1) \text{ op}^a \sigma(e_2)) = \gamma(\sigma(e_1 \text{ op}^s e_2)).$$

We can therefore conclude that $\{\mathcal{S}(e_1 \text{ op}^s e_2)\} \subseteq \gamma(\sigma(e_1 \text{ op}^s e_2))$. □

4 Abstract Interpretation of Programs

4.1 Collecting Denotational Semantics

Let us recall the syntactic definition of our while programs

Aexp $\ni e ::= x \mid n \mid e_1 \text{ op } e_2$

Bexp $\ni b ::= \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 < e_2 \mid b_1 \wedge b_2 \mid \neg b$

While $\ni S ::= x := e \mid \text{skip} \mid S_1; S_2 \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \mid \text{while } b \text{ do } S$

whose denotational semantic functions are:

$$\mathcal{A} : \mathbf{Aexp} \rightarrow \mathbf{State} \rightarrow \mathbb{Z}$$

$$\mathcal{B} : \mathbf{Bexp} \rightarrow \mathbf{State} \rightarrow \mathbb{T}$$

$$\mathcal{S}_{\text{ds}} : \mathbf{While} \rightarrow \mathbf{State} \hookrightarrow \mathbf{State}$$

We define the collecting versions of these semantic functions that are therefore viewed as transformers of state properties in $\wp(\mathbf{State})$.

For arithmetic expressions we have:

$$\mathcal{A}_c : \mathbf{Aexp} \rightarrow \wp(\mathbf{State}) \rightarrow \wp(\mathbb{Z})$$

$$\mathcal{A}_c[e]T \triangleq \{\mathcal{A}[e]s \mid s \in T\}$$

Thus, $\mathcal{A}[e]$ transforms a state property into an integer property.

For Boolean expressions:

$$\begin{aligned}\mathcal{B}_c : \mathbf{Bexp} &\rightarrow \wp(\mathbf{State}) \rightarrow \wp(\mathbf{State}) \\ \mathcal{B}_c[b]T &\triangleq \{s \in T \mid \mathcal{B}[b]s = tt\}\end{aligned}$$

Hence, $\mathcal{B}_c[b]T$ selects those states in T which make the guard b true. $\mathcal{B}_c[b]T$ can also be viewed as a filtering of its argument T .

The collecting semantics of programs $\mathcal{D} : \mathbf{While} \rightarrow \wp(\mathbf{State}) \rightarrow \wp(\mathbf{State})$ is compositionally defined as follows:

$$\begin{aligned}\mathcal{D}[x := e]T &\triangleq \{s[x \mapsto \mathcal{A}[e]s] \in \mathbf{State} \mid s \in T\} \\ \mathcal{D}[\mathbf{skip}]T &\triangleq T \\ \mathcal{D}[S_1; S_2]T &\triangleq (\mathcal{D}[S_2] \circ \mathcal{D}[S_1])T \\ \mathcal{D}[\mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2]T &\triangleq (\mathcal{D}[S_1] \circ \mathcal{B}_c[b])T \cup (\mathcal{D}[S_2] \circ \mathcal{B}_c[\neg b])T\end{aligned}$$

$\mathcal{D}[S]$ can be viewed as a state property transformer where the argument T is a precondition for the program S which is transformed into the strongest postcondition of S .

We need to finalize the above definition with the collecting semantics of while loops. This is defined as a least fixpoint of a continuous function on the complete lattice $\langle \wp(\mathbf{State}), \subseteq \rangle$. For all $T \in \wp(\mathbf{State})$, we define:

$$\mathcal{D}[\mathbf{while } b \mathbf{ do } S]T \triangleq \mathcal{B}_c[\neg b](\text{lfp}(\lambda T'. T \cup (\mathcal{D}[S] \circ \mathcal{B}_c[b])T'))$$

It turns out that the least fixpoint $\text{lfp}(\lambda T'. T \cup (\mathcal{D}[S] \circ \mathcal{B}_c[b])T')$ is the strongest loop invariant. Initially, before evaluating the Boolean guard b , the precondition T holds. Next, an iteration of the while loop for an input condition T' first selects those states in T' that make b true and then applies the body S . This transform $\lambda T'. T \cup (\mathcal{D}[S] \circ \mathcal{B}_c[b])T'$ must be therefore iterated until a fixpoint is reached, which is therefore the strongest loop invariant viewed as state property. Since $\lambda T'. T \cup (\mathcal{D}[S] \circ \mathcal{B}_c[b])T'$ turns out to be continuous, by Kleene-Knaster-Tarski theorem, this latter fixpoint will be the least fixpoint. Finally, the strongest postcondition of the while loop is obtained by filtering the strongest loop invariant with the exit condition $\neg b$.

Let us remark that any $X \in \wp(\mathbf{State})$ which is a fixpoint, i.e., such that $X = T \cup \mathcal{D}[S](\mathcal{B}_c[b]X)$, is a loop invariant, because $T \subseteq X$ and $\mathcal{D}[S](\mathcal{B}_c[b]X) \subseteq X$ both hold.

Example 4.1.1. Consider the program $P \triangleq \mathbf{while } \neg(x = 0) \mathbf{ do } x := x - 2$ and the precondition $T \triangleq \{\{x \rightarrow 5\}, \{x \rightarrow 6\}\} \in \wp(\mathbf{State})$. Then,

$$\begin{aligned}\mathcal{D}[\mathbf{while } \neg(x = 0) \mathbf{ do } x := x - 2]T &= \mathcal{B}_c[\neg \neg(x = 0)] \text{lfp}(F) \quad \text{where} \\ F &\triangleq \lambda T'. T \cup \{s[x \mapsto sx - 2] \mid sx \neq 0, s \in T'\}\end{aligned}$$

Thus, it turns out that:

$$\text{lfp}(F) = \{\dots, \{x \mapsto -5\}, \{x \mapsto -3\}, \{x \mapsto -1\}, \{x \mapsto 0\}, \{x \mapsto 1\}, \\ \{x \mapsto 2\}, \{x \mapsto 3\}, \{x \mapsto 4\}, \{x \mapsto 5\}, \{x \mapsto 6\}\}$$

so that

$$\mathcal{B}_c[\neg\neg(x=0)] \text{lfp}(F) = \{\{x \mapsto 0\}\}.$$

If we execute P with input $x = 5$ then the exit condition $x = 0$ will never be reached, the strongest loop invariant is

$$\{\dots, \{x \mapsto -5\}, \{x \mapsto -3\}, \{x \mapsto -1\}, \{x \mapsto 1\}, \{x \mapsto 3\}, \{x \mapsto 5\}\},$$

and P does not terminate. On the other hand, with input $x = 6$, the strongest loop invariant is $\{x \mapsto 6\}, \{x \mapsto 4\}, \{x \mapsto 2\}, \{x \mapsto 0\}$, and then the execution terminates with output $x = 0$. $\square$

Theorem 4.1.2. *For any $S \in \mathbf{While}$, $T \in \wp(\mathbf{State})$, $\mathcal{D}[S]T = \{S_{\text{ds}}[S]s \in \mathbf{State} \mid s \in T, S_{\text{ds}}[S]s \neq \text{undef}\}$.*

Thus, it turns out that $\mathcal{D}[S]T = \emptyset$ iff, for any $s \in T$, the evaluation of S from s does not terminate.

4.2 Abstract Semantics of Expressions

To design an abstract program semantics we need:

1. an abstraction A of the concrete domain of value properties $\langle \wp(\mathbb{Z}), \subseteq \rangle$;
2. an abstraction $\mathbb{S}$ of the concrete domain of state properties $\langle \wp(\mathbf{State}), \subseteq \rangle$.

We therefore assume that these abstract domains are defined by the GIs $(\alpha_A, \wp(\mathbb{Z}), A, \gamma_A)$ and $(\alpha_{\mathbb{S}}, \wp(\mathbf{State}), \mathbb{S}, \gamma_{\mathbb{S}})$.

4.2.1 Arithmetic Expressions

The abstract semantics of arithmetic expressions is defined by a function

$$\mathcal{A}^\# : \mathbf{Aexp} \rightarrow \mathbb{S} \rightarrow A$$

that abstractly evaluates an expression on a given abstract state by providing as output an abstract value. We require the soundness of this abstract semantics $\mathcal{A}^\#$: for all $e \in \mathbf{Aexp}$ and $s^\# \in \mathbb{S}$,

$$\mathcal{A}_c[e]\gamma_{\mathbb{S}}(s^\#) \subseteq \gamma_A(\mathcal{A}^\#[e]s^\#).$$

The intuition is that the abstract semantics of an expression provides an over-approximation of its concrete collecting semantics. By Galois connection, this soundness requirement can be equivalently stated as follows: for all $e \in \mathbf{Aexp}$, for all $T \in \wp(\mathbf{State})$,

$$\alpha_A(\mathcal{A}_c[e]T) \leq_A \mathcal{A}^\#[e]\alpha_{\mathbb{S}}(T).$$

Here, the approximation comparison between abstract and concrete semantics is done in the abstract domain of values.

4.2.2 Boolean Expressions

The abstract semantics of Boolean expressions is given by

$$\mathcal{B}^\sharp : \mathbf{Bexp} \rightarrow \mathbb{S} \rightarrow \mathbb{S}$$

that must be sound: for all $b \in \mathbf{Bexp}$ and $s^\sharp \in \mathbb{S}$:

$$\mathcal{B}_c[[b]]\gamma_{\mathbb{S}}(s^\sharp) \subseteq \gamma_{\mathbb{S}}(\mathcal{B}^\sharp[[b]]s^\sharp).$$

Similarly to arithmetic expressions, this abstract semantics therefore yields an over-approximation on the concrete filtering semantics of Boolean expressions. Equivalently, by GC, for all $b \in \mathbf{Bexp}$ and $T \in \wp(\mathbf{State})$:

$$\alpha_{\mathbb{S}}(\mathcal{B}_c[[b]]T) \leq_{\mathbb{S}} \mathcal{B}^\sharp[[b]]\alpha_{\mathbb{S}}(T).$$

4.2.3 Abstract State Update

We also need an abstract version of the concrete state update $s[x \mapsto z]$. This is therefore an abstract operation which, given an abstract state $s^\sharp \in \mathbb{S}$, a variable $x \in \mathbf{Var}$, an abstract value $a \in A$, provides an updated abstract state $s^\sharp[x \mapsto a] \in \mathbb{S}$. This update of abstract states must be correct: for all $s^\sharp \in \mathbb{S}$, $x \in \mathbf{Var}$, $a \in A$:

$$\{s[x \mapsto z] \in \mathbf{State} \mid s \in \gamma_{\mathbb{S}}(s^\sharp), z \in \gamma_A(a)\} \subseteq \gamma_{\mathbb{S}}(s^\sharp[x \mapsto a]).$$

The intuition is that if $s^\sharp$ abstracts a state s and a abstracts a value z then $s[x \mapsto z]$ is approximated by $s^\sharp[x \mapsto a]$.

4.3 Abstract Program Semantics

The abstract semantics of programs

$$\mathcal{D}^\sharp : \mathbf{While} \rightarrow \mathbb{S} \rightarrow \mathbb{S}$$

is a transformer of abstract states. This is compositionally defined as follows:

$$\mathcal{D}^\sharp[[x := e]]s^\sharp \triangleq s^\sharp[x \mapsto \mathcal{A}^\sharp[[e]]s^\sharp] \quad (1)$$

$$\mathcal{D}^\sharp[[\mathbf{skip}]]s^\sharp \triangleq s^\sharp \quad (2)$$

$$\mathcal{D}^\sharp[[S_1; S_2]]s^\sharp \triangleq (\mathcal{D}^\sharp[[S_2]] \circ \mathcal{D}^\sharp[[S_1]])s^\sharp \quad (3)$$

$$\mathcal{D}^\sharp[[\mathbf{if} \ b \ \mathbf{then} \ S_1 \ \mathbf{else} \ S_2]]s^\sharp \triangleq (\mathcal{D}^\sharp[[S_1]] \circ \mathcal{B}^\sharp[[b]])s^\sharp \vee_{\mathbb{S}} (\mathcal{D}^\sharp[[S_2]] \circ \mathcal{B}^\sharp[[\neg b]])s^\sharp \quad (4)$$

$$\mathcal{D}^\sharp[[\mathbf{while} \ b \ \mathbf{do} \ S]]s^\sharp \triangleq \mathcal{B}^\sharp[[\neg b]] \left(\text{lfp}(\lambda a. s^\sharp \vee_{\mathbb{S}} (\mathcal{D}^\sharp[[S]] \circ \mathcal{B}^\sharp[[b]])a) \right) \quad (5)$$

Definition (1) leverages the abstract semantics of expressions $\mathcal{A}^\sharp[[e]]s^\sharp$ which is then used for updating the abstract state $s^\sharp$. The soundness of this definition follows from the assumption that the abstract semantics of arithmetic expressions and the update of abstract states are both sound.

The abstract semantics of no-op in (2) is, as expected, the identity on abstract states.

Definition (3) states the analysis propagates forward in the control flow graph of the program. the analysis proceeds by visiting that graph. This definition is correct because it turns out that the composition of abstract functions preserves their soundness.

Definition (4) handles conditional branching. This stipulates that we analyze the true and false branches independently, by composing the abstract semantics of their Boolean guards with the abstract semantics of their bodies. Then, these two abstract states for the true and false branches are joined together through the lub of the abstract domain of states, which is therefore the least common approximation, i.e. lub, of them. Hence, the abstract lub $\vee_{\mathbb{S}}$ can be viewed as a sound approximation of (actually, this turns out to be the best correct approximation of) its concrete counterpart, namely the union of property states, which is their logical disjunction.

The abstract semantics of the while loop in (5) relies on the least fixpoint of the function

$$F_{b,S}^{\sharp} \triangleq \lambda a. s^{\sharp} \vee_{\mathbb{S}} (\mathcal{D}^{\sharp} \llbracket S \rrbracket \circ \mathcal{B}^{\sharp} \llbracket b \rrbracket) a : \mathbb{S} \rightarrow \mathbb{S}$$

which is a transformer of abstract states. This abstract function $F_{b,S}^{\sharp}$ accumulates the abstract information in the loop invariant program point, namely the initial abstract state $s^{\sharp}$ is joined through the lub with the abstract information derived from the analysis of (one iteration of) the body of the loop. A fixpoint of $F_{b,S}^{\sharp}$ —which turns out to be continuous—can be viewed as an abstract invariant of the loop, i.e., it is an (over-)approximation of the strongest concrete loop invariant. Thus, the least fixpoint can be understood as the best approximation derivable in the abstract domain $\mathbb{S}$ of the strongest loop invariant. By assuming that $\langle \mathbb{S}, \leq_{\mathbb{S}} \rangle$, since the abstract transformer turns out to be continuous, this least fixpoint can be characterized by Kleene-Knaster-Tarski theorem:

$$\text{lfp}(\lambda a. s^{\sharp} \vee_{\mathbb{S}} (\mathcal{D}^{\sharp} \llbracket S \rrbracket \circ \mathcal{B}^{\sharp} \llbracket b \rrbracket) a) = \bigvee_{n \in \mathbb{N}} (F_{b,S}^{\sharp})^n(\perp_{\mathbb{S}}).$$

If $\mathbb{S}$ is an ACC CPO then this lub of the Kleene iterates of $F_{b,S}^{\sharp}$ can be reached in finitely many states, and, by assuming that all the abstract operations are computable, therefore provides an effective algorithm. Finally, the abstract post-condition of the while loop is obtained by the abstract filtering of the least fixpoint of $F_{b,S}^{\sharp}$ through the abstract semantics of the exit condition $\neg b$.

Theorem 4.3.1 (Soundness). *For all $S \in \mathbf{While}$, $s^{\sharp} \in \mathbb{S}$:*

$$\mathcal{D} \llbracket S \rrbracket \gamma_{\mathbb{S}}(s^{\sharp}) \subseteq \gamma_{\mathbb{S}}(\mathcal{D}^{\sharp} \llbracket S \rrbracket s^{\sharp}).$$

By Galois connection, the soundness statement can be equivalently given as follows: For all $S \in \mathbf{While}$ and $T \in \wp(\mathbf{State})$,

$$\alpha_{\mathbb{S}}(\mathcal{D} \llbracket S \rrbracket T) \leq_{\mathbb{S}} \mathcal{D}^{\sharp} \llbracket S \rrbracket \alpha_{\mathbb{S}}(T).$$

4.3.1 Fixpoint Soundness and Completeness

The proof of Theorem 4.3.1 crucially relies on the following result.

Lemma 4.3.2 (Fixpoint Soundness). *Let (α, C, A, γ) be a GC between complete lattices, $f : C \rightarrow C$ be a monotonic function, and $f^\sharp : A \rightarrow A$ be a correct approximation of f , i.e., $f(\gamma(a)) \leq_C \gamma(f^\sharp(a))$ for all $a \in A$. Then,*

$$\text{lfp}(f) \leq_C \gamma(\text{lfp}(f^\sharp)).$$

Thus, this result states that the soundness of abstract operations as defined in Section 3.4 can be lifted to fixpoints.

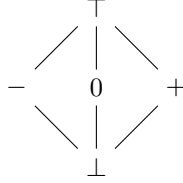
It is also worth remarking that also the completeness of abstract operations (cf. Definition 3.4.3) can be lifted to fixpoints, as stated by the following result.

Lemma 4.3.3 (Fixpoint Completeness). *Let (α, C, A, γ) be a GC between complete lattices, $f : C \rightarrow C$ be a monotonic function, and $f^\sharp : A \rightarrow A$ be a complete approximation of f , i.e., $\alpha \circ f = f^\sharp \circ \alpha$ holds. Then,*

$$\alpha(\text{lfp}(f)) = \text{lfp}(f^\sharp).$$

4.4 Sign Analysis of Programs

Let us recall that the abstract domain Sign is defined by the following Hasse diagram



and represents sign properties of integer program variables. This domain is defined by a GI $(\alpha, \wp(\mathbb{Z}), \text{Sign}, \gamma)$, where, for instance, we have that:

$$\begin{aligned} \alpha(\{-1, -3\}) &= -; & \alpha(\{0, 3\}) &= \top; & \alpha(\{z \in \mathbb{Z} \mid z \geq 3\}) &= +; \\ \gamma(\perp) &= \emptyset; & \gamma(-) &= \{z \in \mathbb{Z} \mid z < 0\}; & \gamma(\top) &= \mathbb{Z}. \end{aligned}$$

This abstraction of integer values induces a corresponding abstraction of program states defined by:

$$\mathbb{S} \triangleq \mathbf{Var} \rightarrow \text{Sign}$$

that is, $\mathbb{S}$ consists of all the maps associating to variables a corresponding abstract sign in Sign . The set $\mathbf{Var}$ is considered to be defined by the set of variables syntactically occurring in the program under analysis, so that $\mathbf{Var}$ is always a finite set. It turns out that $\langle \mathbb{S}, \sqsubseteq, \perp_{\mathbb{S}}, \top_{\mathbb{S}} \rangle$ is a finite lattice (as $\mathbf{Var}$ is a finite set) where:

$$\begin{aligned} \forall s_1^\sharp, s_2^\sharp \in \mathbb{S}. s_1^\sharp \sqsubseteq s_2^\sharp &\iff \forall x \in \mathbf{Var}. s_1^\sharp(x) \leq_{\text{Sign}} s_2^\sharp(x) \\ \perp_{\mathbb{S}} &\triangleq \lambda x. \perp; \\ \top_{\mathbb{S}} &\triangleq \lambda x. \top. \end{aligned}$$

Thus, $s_1^\# \sqsubseteq s_2^\#$ holds when for any variable x , the sign information stored in $s_1^\#$ is more precise (w.r.t. the partial order $\leq_{\text{Sign}}$) of the sign information given by $s_2^\#$. Hence, $\sqsubseteq$ simply lifts in a variable-wise fashion the partial order of Sign.

We define $\alpha_{\mathbb{S}} : \wp(\mathbf{State}) \rightarrow \mathbb{S}$ and $\gamma_{\mathbb{S}} : \mathbb{S} \rightarrow \wp(\mathbb{S})$ as follows:

$$\begin{aligned}\alpha_{\mathbb{S}}(T) &\triangleq \lambda x \in \mathbf{Var}. \alpha(\{s(x) \in \mathbb{Z} \mid s \in T\}) \\ \gamma_{\mathbb{S}}(s^\#) &\triangleq \{s \in \mathbf{State} \mid \forall x \in \mathbf{Var}. \alpha(\{s(x)\}) \leq_{\text{Sign}} s^\#(x)\}\end{aligned}$$

Thus, given a state property T , we collect the values stored in some variable x for some state in T and this value property is abstracted in Sign. On the other hand, an abstract state $s^\#$ represents all the states s such that, for any variable x , the sign of the value stored in x is approximated by the abstract sign $s^\#(x)$. This definition gives rise to a Galois connection.

Lemma 4.4.1. $(\alpha_{\mathbb{S}}, \wp(\mathbf{State}), \mathbb{S}, \gamma_{\mathbb{S}})$ is a GC.

However, we do not have a GI as γ is not injective, as shown by the following example.

Example 4.4.2. We have that:

$$\begin{aligned}\gamma_{\mathbb{S}}(\{x \mapsto -, y \mapsto \perp\}) &= \{s \in \mathbf{State} \mid \alpha(\{s(x)\}) \leq_{\text{Sign}} -, \alpha(\{s(y)\}) = \perp\}; \\ \gamma_{\mathbb{S}}(\{x \mapsto \perp, y \mapsto +\}) &= \{s \in \mathbf{State} \mid \alpha(\{s(x)\}) = \perp, \alpha(\{s(y)\}) \leq_{\text{Sign}} +\}.\end{aligned}$$

Since there is no value $z \in \mathbb{Z}$ such that $\alpha(\{z\}) = \perp$, it turns out that:

$$\gamma_{\mathbb{S}}(\{x \mapsto -, y \mapsto \perp\}) = \emptyset = \gamma_{\mathbb{S}}(\{x \mapsto \perp, y \mapsto +\})$$

namely, γ is not injective. $\square$

The above example hints that as soon as a sign analysis infers that some variable is flagged as $\perp$, the analysis can safely stop and outputs the bottom abstract state $\perp_{\mathbb{S}}$.

4.4.1 Abstract Semantics of Arithmetic Expressions

The abstract semantics $\mathcal{A}^\# : \mathbf{Aexp} \rightarrow \mathbb{S} \rightarrow \text{Sign}$ providing the sign analysis of arithmetic expressions is compositionally defined as follows:

$$\begin{aligned}\mathcal{A}^\#[\![n]\!]s^\# &\triangleq \alpha(\{n\}) \\ \mathcal{A}^\#[\![x]\!]s^\# &\triangleq s^\#(x) \\ \mathcal{A}^\#[\![e_1 \text{ op } e_2]\!]s^\# &\triangleq \mathcal{A}^\#[\![e_1]\!]s^\# \text{ op}^{\text{Sign}} \mathcal{A}^\#[\![e_2]\!]s^\#\end{aligned}$$

where $\text{op}^{\text{Sign}} : \text{Sign} \times \text{Sign} \rightarrow \text{Sign}$ is a correct abstract operation for a binary operation op , with correctness is defined in Section 3.4. For the sake of simplicity, this definition considers binary operations, but, of course, this applies to generic n -ary arithmetic operations.

Example 4.4.3. We have the following abstract semantics:

$$\begin{aligned}\mathcal{A}^\# \llbracket 2 * y + 3 * x \rrbracket \{x \mapsto -, y \mapsto -\} &= -, \\ \mathcal{A}^\# \llbracket x * y - 2 * z \rrbracket \{x \mapsto -, y \mapsto -, z \mapsto +\} &= \top, \\ \mathcal{A}^\# \llbracket x * y - 2 * z \rrbracket \{x \mapsto \perp, y \mapsto -, z \mapsto +\} &= \perp.\end{aligned}\quad \square$$

4.4.2 Abstract Semantics of Boolean Expressions

Let us define the abstract semantics $\mathcal{B}^\# : \mathbf{Bexp} \rightarrow \mathbb{S} \rightarrow \mathbb{S}$ of Boolean expressions in a compositional fashion as follows:

$$\begin{aligned}\mathcal{B}^\# \llbracket true \rrbracket s^\# &\triangleq s^\# \\ \mathcal{B}^\# \llbracket false \rrbracket s^\# &\triangleq \perp_{\mathbb{S}} \\ \mathcal{B}^\# \llbracket e_1 = e_2 \rrbracket s^\# &\triangleq \begin{cases} \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\# \llbracket e_i \rrbracket s^\# \notin \{\top, \perp\}, \mathcal{A}^\# \llbracket e_1 \rrbracket s^\# \neq \mathcal{A}^\# \llbracket e_2 \rrbracket s^\# \\ \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\# \llbracket e_1 \rrbracket s^\# = \perp \text{ or } \mathcal{A}^\# \llbracket e_2 \rrbracket s^\# = \perp \\ s^\# & \text{otherwise} \end{cases} \quad (6)\end{aligned}$$

It must be remarked that definition (6) may not provide the best correct approximation of $\mathcal{B}_c \llbracket e_1 = e_2 \rrbracket$. The first condition of (6) requires that the abstract evaluations of e_1 and e_2 are different signs in $\{-, 0, +\}$, so that, since different signs have an empty intersection (e.g., $\gamma(-) \cap \gamma(+) = \emptyset = \gamma(-) \cap \gamma(0)$), the abstract evaluation of $e_1 = e_2$ can be correctly defined as the bottom abstract state. not $\perp$ or $\top$ and they are not the same) are mutually exclusive. Moreover, the second condition of (6) assumes that either e_1 or e_2 (including both of them) are abstractly evaluated to $\perp$, representing an error, so that the abstract evaluation of $e_1 = e_2$ can be safely defined as bottom. In the third case, the abstract evaluation is simply the identity: it turns out that the identity id is always correct as abstract semantics of any Boolean expression b , because here the requirement is soundness for the collecting filtering function

$$\mathcal{B}_c \llbracket b \rrbracket T \triangleq \{s \in T \mid \mathcal{B} \llbracket b \rrbracket s = tt\}$$

and the containment

$$\mathcal{B}_c \llbracket b \rrbracket \gamma(s^\#) \subseteq \gamma(s^\#) = \gamma(\text{id}(s^\#)) = \gamma(\mathcal{B}^\# \llbracket b \rrbracket s^\#)$$

always holds. However, this is a correct approximation which may well not be the best, as shown by the following example.

Example 4.4.4. We have that

$$\begin{aligned}\mathcal{B}_c \llbracket x + 1 = x \rrbracket \gamma_{\mathbb{S}}(\{x \mapsto +\}) &= \mathcal{B}_c \llbracket x + 1 = x \rrbracket \{s \in \mathbf{State} \mid s(x) > 0\} = \\ &= \{s \in \mathbf{State} \mid s(x) > 0, s(x) + 1 = s(x)\} = \emptyset\end{aligned}$$

while, according to definition (6),

$$\mathcal{B}^\# \llbracket x + 1 = x \rrbracket \{x \mapsto +\} = \{x \mapsto +\}$$

because $\mathcal{A}^\# \llbracket x + 1 \rrbracket \{x \mapsto +\} = +$ and $\mathcal{A}^\# \llbracket x \rrbracket \{x \mapsto +\} = +$. Thus, in this case, definition (6) does not coincide with the best correct approximation

$$\alpha_{\mathbb{S}}(\mathcal{B}_c \llbracket x + 1 = x \rrbracket \gamma_{\mathbb{S}}(\{x \mapsto +\})) = \perp_{\mathbb{S}}. \quad \square$$

Let us continue the compositional definition of $\mathcal{B}^\sharp$ as follows:

$$\mathcal{B}^\sharp[e_1 \neq e_2]s^\sharp \triangleq \begin{cases} \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\sharp[e_1]s^\sharp = 0 = \mathcal{A}^\sharp[e_2]s^\sharp \\ \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\sharp[e_1]s^\sharp = \perp \text{ or } \mathcal{A}^\sharp[e_2]s^\sharp = \perp \\ s^\sharp & \text{otherwise} \end{cases}$$

$$\mathcal{B}^\sharp[e_1 < e_2]s^\sharp \triangleq \begin{cases} \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\sharp[e_i]s^\sharp \notin \{\perp, \top\}, \mathcal{A}^\sharp[e_1]s^\sharp \succ \mathcal{A}^\sharp[e_2]s^\sharp \\ \perp_{\mathbb{S}} & \text{if } \mathcal{A}^\sharp[e_1]s^\sharp = \perp \text{ or } \mathcal{A}^\sharp[e_2]s^\sharp = \perp \\ s^\sharp & \text{otherwise} \end{cases} \quad (7)$$

$$\mathcal{B}^\sharp[e_1 \leq e_2]s^\sharp \triangleq \mathcal{B}^\sharp[e_1 < e_2]s^\sharp$$

$$\mathcal{B}^\sharp[b_1 \wedge b_2]s^\sharp \triangleq (\mathcal{B}^\sharp[b_2] \circ \mathcal{B}^\sharp[b_1])s^\sharp \quad (8)$$

$$\mathcal{B}^\sharp[b_1 \vee b_2]s^\sharp \triangleq \mathcal{B}^\sharp[b_1]s^\sharp \vee_{\mathbb{S}} \mathcal{B}^\sharp[b_2]s^\sharp \quad (9)$$

In the first condition of definition (7), where $\mathcal{A}^\sharp[e_i]s^\sharp \in \{-, 0, +\}$, the meaning of the relation $\succ$ on $\{-, 0, +\}$ is as expected: $+$ $\succ$ 0 , 0 $\succ$ $-$; that is, positive is “greater” than zero, and zero is “greater” than negative. Moreover, in (8), logical conjunction is abstractly evaluated as composition of the corresponding abstract semantics, while in (9), disjunction boils down to the least upper bound between abstract states.

Example 4.4.5. According to the above definitions, we have that:

$$\begin{aligned} \mathcal{B}^\sharp[x < -1]\{x \mapsto +\} &= \perp_{\mathbb{S}} \\ \mathcal{B}^\sharp[x \neq 2 * y]\{x \mapsto +, y \mapsto +\} &= \{x \mapsto +, y \mapsto +\} \\ \mathcal{B}^\sharp[x = 2 * y \vee x < -1]\{x \mapsto +, y \mapsto +\} &= \{x \mapsto +, y \mapsto +\} \vee_{\mathbb{S}} \perp_{\mathbb{S}} \\ &= \{x \mapsto +, y \mapsto +\} \quad \square \end{aligned}$$

Of course, this definition of $\mathcal{B}^\sharp$ can be made much more precise. For instance, we should aim at a definition such that

$$\mathcal{B}^\sharp[x < y]\{x \mapsto \top, y \mapsto 0\} = \{x \mapsto -, y \mapsto 0\}$$

holds.

4.4.3 Abstract Update

The abstract update $(\cdot)[(\cdot) \mapsto (\cdot)] : \mathbb{S} \times \mathbf{Var} \times \text{Sign} \rightarrow \mathbb{S}$ is defined as follows:

$$s^\sharp[x \mapsto a] \triangleq \lambda y. \begin{cases} a & \text{if } y = x \\ s^\sharp(y) & \text{if } y \neq x \end{cases} \quad (10)$$

Thus, if $y = x$ then the sign of the variable y is updated and the other variables are left unchanged. This is the best approximation we can make because the abstract states are updated variable by variable. It is worth remarking that this definition (10) is not specifically tailored for Sign but can be applied to any domain A abstracting $\wp(\mathbb{Z})$.

4.4.4 An Example of Sign Analysis

Let us consider the program

$$P \triangleq \mathbf{while} \ x \neq 0 \ \mathbf{do} \ x := x + 1$$

whose abstract semantics is:

$$\mathcal{D}^\# \llbracket P \rrbracket s^\# = \mathcal{B}^\# \llbracket x = 0 \rrbracket \left(\text{lfp}(\lambda t^\#. s^\# \vee_{\mathbb{S}} (\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) t^\#) \right)$$

Consider $s_1^\# = \{x \mapsto -\}$. We have that:

$$(\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) s_1^\# = \mathcal{D}^\# \llbracket x := x + 1 \rrbracket s_1^\# = \{x \mapsto \top\} \quad (11)$$

$$s_1^\# \vee_{\mathbb{S}} \{x \mapsto \top\} = \{x \mapsto \top\}$$

$$\text{lfp}(\lambda t^\#. s_1^\# \vee_{\mathbb{S}} (\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) t^\#) = \{x \mapsto \top\} \quad (12)$$

In turn, we obtain

$$\mathcal{D}^\# \llbracket P \rrbracket s^\# = \mathcal{B}^\# \llbracket x = 0 \rrbracket \{x \mapsto \top\} = \{x \mapsto 0\}.$$

Observe that (11) is justified by the fact that $\mathcal{A}^\# \llbracket x + 1 \rrbracket s_1^\# = \top$. Moreover, since $\{x \mapsto \top\}$ is the top of $\mathbb{S}$ then in (12) it is trivially the least fixpoint. Thus, this analysis of P carries no interesting information: it simply tells us that if we execute P with a negative input value for x and P terminates then x will store the value 0; but this information is obvious because $x = 0$ is exit condition.

Consider now $s_2^\# = \{x \mapsto 0\}$. In this case we have that:

$$(\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) s_2^\# = \mathcal{D}^\# \llbracket x := x + 1 \rrbracket \{x \mapsto \perp\} = \{x \mapsto \perp\}$$

$$s_2^\# \vee_{\mathbb{S}} \{x \mapsto \perp\} = \{x \mapsto 0\}$$

$$\text{lfp}(\lambda t^\#. s_2^\# \vee_{\mathbb{S}} (\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) t^\#) = \{x \mapsto 0\}$$

$$\mathcal{B}^\# \llbracket x = 0 \rrbracket \{x \mapsto 0\} = \{x \mapsto 0\}$$

Hence, also in this case the analysis of P infers just trivial information.

Finally, let us consider $s_3^\# = \{x \mapsto +\}$.

$$(\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) s_3^\# = \mathcal{D}^\# \llbracket x := x + 1 \rrbracket \{x \mapsto +\} = \{x \mapsto +\}$$

$$s_3^\# \vee_{\mathbb{S}} \{x \mapsto +\} = \{x \mapsto +\}$$

$$\text{lfp}(\lambda t^\#. s_3^\# \vee_{\mathbb{S}} (\mathcal{D}^\# \llbracket x := x + 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \neq 0 \rrbracket) t^\#) = \{x \mapsto +\}$$

Hence, here we obtain

$$\mathcal{D}^\# \llbracket P \rrbracket s^\# = \mathcal{B}^\# \llbracket x = 0 \rrbracket \{x \mapsto +\} = \{x \mapsto \perp\}.$$

This output abstract state is obtained because a positive number can never be zero, so that the abstract semantics of the $x = 0$ can infer the bottom state $\perp$ representing unreachability. Here, the analysis is able to derive some nontrivial information: if we execute P with a positive input value for x then P definitely does not terminate.

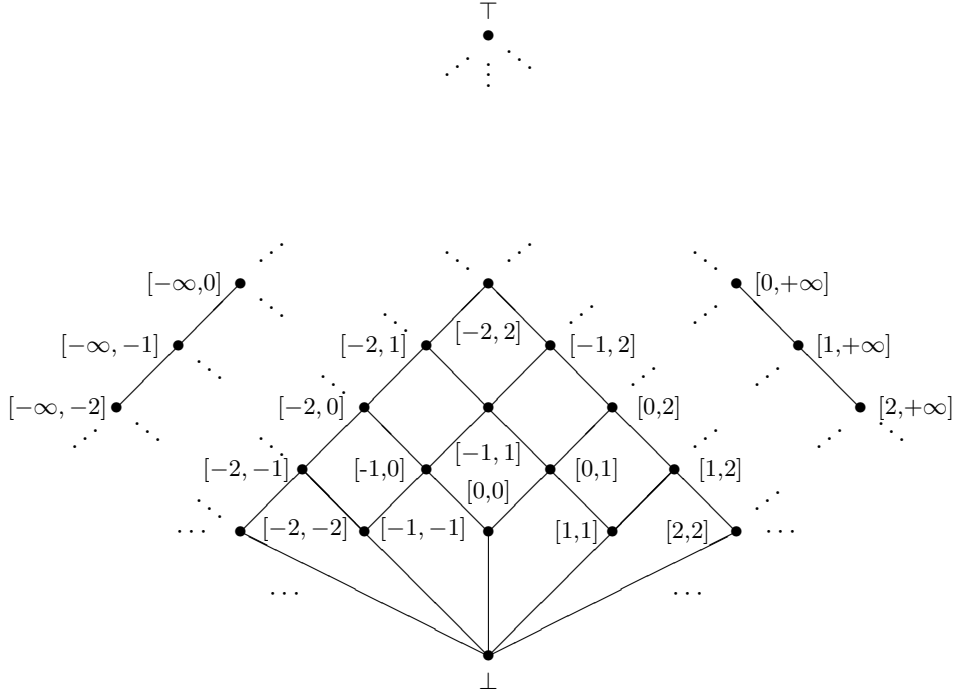


Figure 8: The Interval Abstract Domain Int.

4.5 Interval Analysis of Programs

We consider the complete lattice Int of integer intervals depicted in Figure 8. We extend integer numbers to $\mathbb{Z}_\infty \triangleq \mathbb{Z} \cup \{-\infty, +\infty\}$ where for all $z \in \mathbb{Z}_\infty$, $-\infty \leq z \leq +\infty$ holds. Thus,

$$\text{Int} \triangleq \{[l, u] \mid l, u \in \mathbb{Z}_\infty, l \leq u\} \cup \{\perp\},$$

where $[-\infty, +\infty]$ is also denoted by $\top$. The intuition is that an interval $[l, u]$ represents a range of variation for an integer program variable, where l is a lower bound and u is an upper bound for the values that can be stored in that variable. The ordering $\preceq$ of Int is defined as follows:

$$\perp \preceq [l, u]; \quad [l_1, u_1] \preceq [l_2, u_2] \Leftrightarrow l_2 \leq l_1 \ \& \ u_1 \leq u_2$$

It turns out that $\langle \text{Int}, \preceq \rangle$ is a complete lattice whose glb $\wedge$ is given by intersection. Int is defined by a GI $(\alpha, \wp(\mathbb{Z}), \text{Int}, \gamma)$ GI where:

$$\begin{aligned} \gamma(\perp) &\triangleq \emptyset; & \gamma([l, u]) &\triangleq \{x \in \mathbb{Z} \mid l \leq x \leq u\}; \\ \alpha(X) &\triangleq \begin{cases} \wedge \{[l, u] \in \text{Int} \mid X \subseteq \gamma([l, u])\} & \text{if } X \neq \emptyset, \\ \perp & \text{if } X = \emptyset. \end{cases} \end{aligned}$$

For example, we have that

$$\begin{aligned}\alpha(\{-1, 2, 5\}) &= [-1, 5], \\ \alpha(\{z \in \mathbb{Z} \mid z \text{ is even}\}) &= \top, \\ \alpha(\{z \in \mathbb{Z} \mid z \geq 3, z \neq 9\}) &= [3, +\infty).\end{aligned}$$

A corresponding domain of abstract states is defined variable-wise as follows:

$$\mathbb{S} \triangleq \mathbf{Var} \rightarrow \text{Int}$$

where for all $s^\sharp, t^\sharp \in \mathbb{S}$, $s^\sharp \sqsubseteq t^\sharp$ iff for all $x \in \mathbf{Var}$, $s^\sharp(x) \preceq t^\sharp(x)$.

4.5.1 An Example of Interval Analysis

In these notes, we will not provide an explicit and precise definition of the main abstract operations on Int . In the following example, we will simply use what appears to be the natural best approximation in the domain of intervals of abstract operations.

We consider the program

$$P \triangleq \mathbf{while} \ x \geq 0 \ \mathbf{do} \ \{x := x - 1; y := y + 1\}$$

whose abstract semantics is therefore:

$$\begin{aligned}\mathcal{D}^\sharp[P]s^\sharp &\triangleq \\ \mathcal{B}^\sharp[x < 0] \Big(\text{lfp}(\lambda t^\sharp. s^\sharp \sqcup_{\mathbb{S}} (\mathcal{D}^\sharp[y := y + 1] \circ \mathcal{D}^\sharp[x := x - 1] \circ \mathcal{B}^\sharp[x \geq 0])t^\sharp) \Big)\end{aligned}$$

Let us consider as input abstract state $s^\sharp \triangleq \{x \mapsto [10, 10], y \mapsto [0, 0]\}$. Let us compute the Kleene iteration sequence of the function

$$F \triangleq \lambda t^\sharp. s^\sharp \sqcup_{\mathbb{S}} (\mathcal{D}^\sharp[y := y + 1] \circ \mathcal{D}^\sharp[x := x - 1] \circ \mathcal{B}^\sharp[x \geq 0])t^\sharp : \text{Int} \rightarrow \text{Int}.$$

$$F^0(\perp) = \perp$$

$$F^1(\perp) = s^\# \sqcup_{\mathbb{S}} \perp = s^\#$$

$$\begin{aligned} F^2(\perp) &= s^\# \sqcup_{\mathbb{S}} (\mathcal{D}^\# \llbracket y := y + 1 \rrbracket \circ \mathcal{D}^\# \llbracket x := x - 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \geq 0 \rrbracket) s^\# \\ &= s^\# \sqcup_{\mathbb{S}} (\mathcal{D}^\# \llbracket y := y + 1 \rrbracket \circ \mathcal{D}^\# \llbracket x := x - 1 \rrbracket) \{x \mapsto [10, 10], y \mapsto [0, 0]\} \\ &= s^\# \sqcup_{\mathbb{S}} \mathcal{D}^\# \llbracket y := y + 1 \rrbracket \{x \mapsto [9, 9], y \mapsto [0, 0]\} \\ &= s^\# \sqcup_{\mathbb{S}} \{x \mapsto [9, 9], y \mapsto [1, 1]\} \\ &= \{x \mapsto [9, 10], y \mapsto [0, 1]\} \end{aligned}$$

$$\begin{aligned} F^3(\perp) &= s^\# \sqcup_{\mathbb{S}} (\mathcal{D}^\# \llbracket y := y + 1 \rrbracket \circ \mathcal{D}^\# \llbracket x := x - 1 \rrbracket \circ \mathcal{B}^\# \llbracket x \geq 0 \rrbracket) \{x \mapsto [9, 10], y \mapsto [0, 1]\} = \\ &= s^\# \sqcup_{\mathbb{S}} (\mathcal{D}^\# \llbracket y := y + 1 \rrbracket \circ \mathcal{D}^\# \llbracket x := x - 1 \rrbracket) \{x \mapsto [9, 10], y \mapsto [0, 1]\} = \\ &= s^\# \sqcup_{\mathbb{S}} \mathcal{D}^\# \llbracket y := y + 1 \rrbracket \{x \mapsto [8, 9], y \mapsto [0, 1]\} \\ &= s^\# \sqcup_{\mathbb{S}} \{x \mapsto [8, 9], y \mapsto [1, 2]\} \\ &= \{x \mapsto [8, 10], y \mapsto [0, 2]\} \end{aligned}$$

Hence, the Kleene iteration sequence $\{F^n(\perp)\}_{n \in \mathbb{N}}$ does not finitely converge, because for all $n \geq 12$ it turns out that

$$F^n(\perp) = \{x \mapsto [-1, 10], y \mapsto [0, n-1]\},$$

so that the lub in $\mathbb{S}$ of this infinite increasing chain $\{F^n(\perp)\}_{n \in \mathbb{N}}$ turns out to be:

$$\text{lfp}(F) = \{x \mapsto [-1, 10], y \mapsto [0, +\infty]\}.$$

which is the abstract loop invariant of P . Clearly, the problem is that while the interval for the variable x is lower bounded by the Boolean guard $x \geq 0$, no upper bound can be inferred for the variable y , so that the abstract semantics infers as abstract invariant for y the interval $[-, +\infty]$.

As abstract post-condition we therefore get:

$$\begin{aligned} \mathcal{D}^\# \llbracket P \rrbracket s^\# &= \mathcal{B}^\# \llbracket x < 0 \rrbracket \{x \mapsto [-1, 10], y \mapsto [0, +\infty]\} \\ &= \{x \mapsto [-1, -1], y \mapsto [0, +\infty]\} \end{aligned}$$

Hence, in this case, it turns out that the analysis of P is not terminating. We therefore need a technique to extrapolate, in a *finite number* of steps, the interval $[0, +\infty]$ for the variable y .

4.6 Widening

The notion of widening is central in abstract interpretation.

Definition 4.6.1. A binary operator $\nabla : L \times L \rightarrow L$, commonly used in infix notation, on a complete lattice $(L, \leq_L)$ is called a *widening* if:

- (1) it is a finite upper bound, i.e., for all $x, y \in L$, $x, y \leq_L x \nabla y$
- (2) it enforces convergence, i.e., for any ascending chain $(x_n)_{n \in \mathbb{N}}$ in L , the induced ascending chain $(y_n)_{n \in \mathbb{N}}$, inductively defined by:

$$y_0 \triangleq x_0, \quad y_{n+1} \triangleq y_n \nabla x_{n+1},$$

is not strictly increasing, i.e., $(y_n)_{n \in \mathbb{N}}$ finitely converges. $\square$

Let us stress that the y_{n+1} is defined by using the predecessor element y_n which is widened with the corresponding element x_{n+1} of the original chain $(x_n)_{n \in \mathbb{N}}$.

Let us consider the complete lattice of integer intervals Int whose binary lub γ turns out to be as follows:

$$[l_1, u_1] \gamma [l_2, u_2] \triangleq [\min(l_1, l_2), \max(u_1, u_2)].$$

Let $K \in \text{Int}$ be a given interval and let us define a binary operator $\Omega_K : \text{Int} \times \text{Int} \rightarrow \text{Int}$ as follows:

$$\Omega_K([l_1, u_1], [l_2, u_2]) \triangleq \begin{cases} [l_1, u_1] \gamma [l_2, u_2] & \text{if } [\min(l_1, l_2), \max(u_1, u_2)] \preceq K \\ [-\infty, +\infty] & \text{otherwise} \end{cases}$$

Thus, if the lub $[l_1, u_1] \gamma [l_2, u_2] = [\min(l_1, l_2), \max(u_1, u_2)]$ is included in K then the operator Ω_K acts as the lub of Int , otherwise it widens this lub to the top interval $[-\infty, +\infty]$.

Example 4.6.2. Let $K = [0, +\infty] \in \text{Int}$. The following example shows that $\Omega_{[0, +\infty]}$ is not a widening. In fact, the transform of Definition 4.6.1 (2) leaves the infinite ascending chain $([0, n])_{n \in \mathbb{N}}$ unchanged, because for all n ,

$$\Omega_{[0, +\infty]}([0, n], [0, n+1]) = [0, n+1].$$

Hence, this identity transform of the chain does not finitely converge, meaning that $\Omega_{[0, +\infty]}$ does not satisfy the requirement of Definition 4.6.1 (2) (requirement (1) is instead satisfied). $\square$

It turns out that if K is a finite interval then Ω_K is a widening.

Example 4.6.3. Let us consider $K = [-2, 4] \in \text{Int}$. The operator $\Omega_{[-2, 4]}$ turns out to be a widening. For instance, the diverging chain $([0, n])_{n \in \mathbb{N}}$ is transformed by Definition 4.6.1 (2) into the increasing chain

$$[0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [-\infty, +\infty], [-\infty, +\infty], \dots$$

that finitely converges to $[-\infty, +\infty]$. $\square$

4.6.1 Standard Interval Widening

The standard widening $\nabla : \text{Int} \times \text{Int} \rightarrow \text{Int}$ on intervals is defined as follows:

$$\begin{aligned} \perp \nabla [l, u] &\triangleq [l, u] \\ [l, u] \nabla \perp &\triangleq [l, u] \\ [l_1, u_1] \nabla [l_2, u_2] &\triangleq [\text{if } l_1 \leq l_2 \text{ then } l_1 \text{ else } -\infty, \text{if } u_2 \leq u_1 \text{ then } u_1 \text{ else } +\infty] \end{aligned}$$

Thus, with this standard widening the unstable left (i.e., $l_2 < l_1$) and right (i.e., $u_1 < u_2$) bounds are widened, respectively, to $-\infty$ and $+\infty$.

For instance, we have that:

$$[0, 1] \subseteq [0, 2] \quad \text{but} \quad [0, 1] \nabla [0, 2] = [0, +\infty] \not\subseteq [0, 2] = [0, 2] \nabla [0, 2], \quad (13)$$

$$[0, 2] \nabla [0, 1] = [0, 2] \quad \text{but} \quad [0, 1] \nabla [0, 2] = [0, +\infty]. \quad (14)$$

Hence, it turns out that this standard widening is neither monotonic in its first argument (by example (13)) nor commutative (by example (14)).

Let f be a continuous function $f : L \rightarrow L$ on a complete lattice $\langle L, \leq \rangle$ and consider a widening ∇ on L . We inductively define the following *widened Kleene iteration sequence* $(x^n)_{n \in \mathbb{N}}$ induced by ∇ :

$$\begin{aligned} x^0 &\triangleq \perp \\ x^{n+1} &\triangleq \begin{cases} x^n & \text{if } f(x^n) \leq x^n \\ x^n \nabla f(x^n) & \text{if } f(x^n) \not\leq x^n \end{cases} \end{aligned}$$

Therefore, in the inductive step, if the application of f to the previous element of the chain x^n is a prefix point (i.e., $f(x^n)$ is bounded by x^n) then we keep x^n as next element and therefore the chain is converging, otherwise we consider the widening of x^n with $f(x^n)$. The following result can be proved.

Theorem 4.6.4. *The ascending chain $\{x^n\}_{n \in \mathbb{N}}$ finitely converges to an element $z \in L$ such that: (1) $f(z) \leq z$ and (2) $\text{lfp}(f) \leq z$ hold.*

Thus, the widened Kleene iteration sequence is guaranteed to finitely converge to an element which is prefix point and an over-approximation of the least fixpoint of f .

4.6.2 An Example of Interval Analysis with Widening

Let us consider the program:

$$P \triangleq \text{int } x := 1; \text{ while } x \leq 100 \text{ do } x := x + 1$$

The interval analysis $\mathcal{D}^\sharp[[P]]^\top$ boils down to compute the least fixpoint of the function $F_P : \text{Int} \rightarrow \text{Int}$ defined as follows:

$$F_P(I) \triangleq [1, 1] \vee ((I \wedge [-\infty, 100]) \oplus [1, 1])$$

where: $[1, 1]$ is the interval after the initialization $\mathbf{int} \ x := 1$; $I \wedge [-\infty, 100]$ is the input interval I filtered by the Boolean guard $x \leq 100$; $I_1 \oplus I_2$ is the binary addition of two intervals.

We compute the widened Kleene iteration sequence $(I_n)_{n \in \mathbb{N}}$ of F_P leveraging the standard widening defined in Section 4.6.1. By Theorem 4.6.4 we know that this widened sequence finitely converges to an interval that overapproximates $\text{lfp}(F_P)$.

$$\begin{aligned}
I_0 &= \perp \\
I_1 &= I_0 \nabla F_P(I_0) = \perp \nabla ([1, 1] \vee ((\perp \wedge [-\infty, 100]) \oplus [1, 1])) \\
&= \perp \nabla ([1, 1] \vee (\perp \oplus [1, 1])) = \perp \nabla ([1, 1] \vee \perp) = \perp \nabla [1, 1] = [1, 1] \\
I_2 &= I_1 \nabla F_P(I_1) = [1, 1] \nabla ([1, 1] \vee (([1, 1] \wedge [-\infty, 100]) \oplus [1, 1])) \\
&= [1, 1] \nabla ([1, 1] \vee ([1, 1] \oplus [1, 1])) = [1, 1] \nabla ([1, 1] \vee [2, 2]) \\
&= [1, 1] \nabla [1, 2] = [1, +\infty] \\
I_3 &= I_2 \nabla F_P(I_2) = [1, +\infty] \nabla ([1, 1] \vee (([1, +\infty] \wedge [-\infty, 100]) \oplus [1, 1])) \\
&= [1, +\infty] \nabla ([1, 1] \vee ([1, 100] \oplus [1, 1])) = [1, +\infty] \nabla ([1, 1] \vee [2, 101]) \\
&= [1, +\infty] \nabla [1, 101] = [1, +\infty] = I_2
\end{aligned}$$

Hence, in four iterations, the widened Kleene iteration sequence converges to the interval $[1, +\infty]$, which is an approximation of the true least fixpoint $\text{lfp}(F_P) = [1, 101]$, that actually is the strongest loop invariant of the while-loop with precondition $[1, 1]$. This allows us to conclude that

$$\mathcal{D}^\sharp \llbracket P \rrbracket \top = \mathcal{B}^\sharp \llbracket x > 100 \rrbracket [1, +\infty] = [101, +\infty] \wedge [1, +\infty] = [101, +\infty]$$

namely, $x > 100$ is a correct post-condition for P .

Bibliography

- P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. ACM POPL'77*, pages 238–252. ACM, 1977. doi: 10.1145/512950.512973. URL <https://doi.org/10.1145/512950.512973>.
- A. Miné. *Weakly Relational Numerical Abstract Domains*. PhD thesis, École Normale Supérieure, Paris, France, 2004.